

محمدرضا شریفی

۴۰۱۱۰۱۹۳۲

گزارش پروژه نهایی

درس سیستم های نهفته بی درنگ

پیش از شروع لازم به ذکر است تست های انجام شده برای هر بخش گام به گام انجام شده و به همین علت خروجی تصاویر آورده شده مربوط به میزان کد نوشته در همان بخش هستند و در هر مرحله کد ها دستخوش تغییرات شدند و در نتیجه خروجی ها نیز کمی دستخوش تغییر شدند اما backward compatibility کاملاً حفظ شده و نسخه نهایی در سامانه درس بارگذاری شده است.

بخش اول: طراحی وب سرور، سرویس خودکار و امن سازی با SSL

الف) طراحی و پیاده سازی وب سرور HTTP/HTTPS

- در این پروژه برای به حداقل رساندن سربار پردازشی روی برد و پیاده سازی یک سامانه نهفته بلا درنگ، منطق اصلی وب سرور مستقیماً با زبان C و برنامه نویسی سوکت (Socket Programming) توسعه داده شده است.
- **میزبانی صفحه HTML:** در پاسخ به درخواست های روت آدرس اصلی سرور، یک صفحه وب به زبان HTML تولید و ارسال می شود. عنوان (Title) این صفحه شامل نام و شماره دانشجویی (Mohammadreza Sharifi - 401101932) است تا هویت توسعه دهنده سیستم تایید شود.
 - **استریم زنده تصویر:** از آنجا که پردازش تصویر و اتصال دوربین به صورت مجزا در یک اسکریپت پایتون (لپ تاپ) انجام می شود، خروجی این استریم از طریق تگ در واسط کاربری وب سرور C به صورت زنده نمایش داده می شود.
 - **نمایش زنده اطلاعات تله متری:** وضعیت لحظه ای سیستم شامل دمای پردازنده، حافظه آزاد و درصد مصرف پردازنده به صورت مستقیم از طریق خواندن فایل های سیستمی لینوکس (sys/class/thermal/thermal_zone0/temp/) برای proc/stat/ و proc/meminfo/ و در رابط کاربری نمایش داده می شوند.
- قطعه کد تولید صفحه وب در زبان C:**

```
441 char html_response[2500];
442 sprintf(html_response,
443 "HTTP/1.1 200 OK\r\nContent-Type: text/html\r\nConnection: close\r\n\r\n"
444 "<html><head><title>MohammadrezaSharifi - 401101932</title>"
445 "<script>function toggleGuard(){\n"
446 "var newState = %d == 1 ? 0 : 1;\n"
447 "fetch('/api/v1/guard', {method: 'POST', body: JSON.stringify({state: newState})});\n"
448 ".then(() => location.reload());}</script>"
449 "</head>"
450 "<body style='font-family: Arial, sans-serif; text-align: center; background-color: #f4f4f4;'\n"
451 "<h2>Smart Security System (HTTPS)</h2>"
452 "<img src='http://s:5000/video_feed' width='640' height='480' style='border: 2px solid black;' />"
453 "<div style='margin-top: 20px; font-size: 1.2em;'\n"
454 "<p><b>CPU Temp:</b> %1f &deg;C | <b>CPU Load:</b> %1f%% | <b>Free RAM:</b> %1d kB</p>"
455 "<p><b>Persons Detected:</b> %d</p>"
456 "<button onclick='toggleGuard()' style='padding:10px; font-size:16px; background-color:%s; color:white; border:none; border-radius:5px;'\n"
457 "%s</button>"
458 "</div></body></html>",
459 guard_mode, client_ip, get_cpu_temp(), get_cpu_load(), get_free_memory(), current_persons,
460 guard_mode == 1 ? "red" : "green", guard_mode == 1 ? "Guard Mode ACTIVE (Click to Disable)" : "Guard Mode OFF (Click to Enable)");
461 SSL_write(ssl, html_response, strlen(html_response));
462
```

ب) پیاده سازی الزامات امنیتی و گواهی SSL جهت تامین امنیت و جلوگیری از شنود داده ها، ارتباطات سامانه بر بستر پروتکل HTTPS ایمن شده اند:

- **تولید گواهی Self-Signed:** با استفاده از ابزار openssl یک گواهی امنیتی اختصاصی تولید شد. در این گواهی فیلد CN یا همان Common Name دقیقاً برابر با شماره دانشجویی (۴۰۱۱۰۱۹۳۲) تنظیم شده است.
- **تغییر مسیر (Redirect):** برای جلوگیری از اتصالات ناامن، یک Thread مجزا به نام http_redirect_thread روی پورت ۸۰۸۰ اجرا شده است که هرگونه درخواست http را با کد وضعیت ۳۰۱ Moved Permanently به پورت ۸۴۴۳ (HTTPS) هدایت می کند.

قطعه کد هدایت خودکار (Redirect):

```
266 while (1) {
267     if ((client_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t*)&addrlen)) >= 0) {
268         char buffer[1024] = {0};
269         read(client_socket, buffer, sizeof(buffer));
270         struct sockaddr_in local_addr;
271         socklen_t local_addr_len = sizeof(local_addr);
272         getsockname(client_socket, (struct sockaddr *)&local_addr, &local_addr_len);
273         char redirect_response[512];
274         sprintf(redirect_response,
275             "HTTP/1.1 301 Moved Permanently\r\nLocation: https://%s:%d/\r\nContent-Length: 0\r\nConnection: close\r\n\r\n",
276             inet_ntoa(local_addr.sin_addr), HTTPS_PORT);
277         write(client_socket, redirect_response, strlen(redirect_response));
278         close(client_socket);
279     }
280 }
```

ج) اجرای خودکار و مدیریت سرویس‌ها با **Systemd** و گراف وابستگی برای تضمین اجرای خودکار و بلادرنگ سامانه پس از روشن شدن دستگاه، از قابلیت‌های **systemd** استفاده شده است. سرویس **webserver.service** وظیفه مدیریت وب‌سرور **C** را بر عهده دارد.

- مدیریت وابستگی سرویس‌ها (**Service Dependencies**): از آنجا که ماژول **API Gateway** (مبتنی بر **FastAPI**/پایتون) تنها یک لایه مستندساز و واسط است، حتماً باید پس از اجرای کامل سرور **C** بالا بیاید. به همین منظور در پیکربندی سرویس **apigateway.service** از دستورات **After=webserver.service** و **Requires=webserver.service** استفاده شده است. این وابستگی تضمین می‌کند که سیستم‌عامل ابتدا هسته اصلی سامانه (سرور **C**) را در مدار قرار دهد و در صورت توقف یا ری‌استارت شدن سرور **C**، سرویس **Gateway** نیز به تبعیت از آن مدیریت شود.

سرویس **webserver.service**:

```
1 [Unit]
2 Description=Smart Security HTTPS Web Server
3 After=network-online.target
4 Wants=network-online.target
5
6 [Service]
7 Type=simple
8 User=guard
9 WorkingDirectory=/home/guard
10 ExecStart=/home/guard/webserver
11 Restart=always
12 RestartSec=3s
13
14 [Install]
15 WantedBy=multi-user.target
```

سرویس **apigateway.service**:

```
1 [Unit]
2 Description=FastAPI Swagger Gateway for Smart Security
3 After=network-online.target webserver.service
4 Requires=webserver.service
5
6 [Service]
7 Type=simple
8 User=guard
9 WorkingDirectory=/home/guard
10 # We use python3 -m uvicorn so we don't have to guess where the uvicorn binary is installed
11 ExecStart=/usr/bin/python3 -m uvicorn api_gateway:app --host 0.0.0.0 --port 8000
12 Restart=always
13 RestartSec=5s
14
15 [Install]
16 WantedBy=multi-user.target
```

آزمایش‌های الزامی بخش اول

آزمایش ۱-۱ بررسی زمان بوت: با توجه به اینکه نوع سرویس ما Type=simple است، استفاده از دستور `systemd-analyze blame` به تنهایی معیار دقیقی برای سنجش زمان بالا آمدن کامل این سرویس ارائه نمی‌دهد. بنابراین برای تحلیل دقیق‌تر از دستور `systemd-analyze critical-chain webserver.service` استفاده شد. نتایج نشان می‌دهد که سرویس وب‌سرور پس از گذشت حدود ۹.۲۴۹ ثانیه از زمان شروع بوت سیستم، به طور کامل در مدار قرار گرفته است. بررسی وضعیت با `systemctl status` نیز پایداری سرویس را تایید می‌کند.

```
guard@orangezipzeroplus2h5:~$ sudo systemctl status webserver.service
[sudo] password for guard:
● webserver.service - Smart Security HTTPS Web Server
   Loaded: loaded (/etc/systemd/system/webserver.service; enabled; vendor pre>
   Active: active (running) since Thu 2026-07-23 13:24:02 UTC; 24h ago
   Main PID: 949 (webserver)
     Tasks: 2 (limit: 403)
    Memory: 1.0M
   CGroup: /system.slice/webserver.service
           └─949 /home/guard/webserver

Jul 23 13:24:02 orangezipzeroplus2h5 systemd[1]: Started Smart Security HTTPS We>
guard@orangezipzeroplus2h5:~$ systemd-analyze critical-chain webserver.service
The time when unit became active or started is printed after the "@" character.
The time the unit took to start is printed after the "+" character.

webserver.service @9.249s
└─network-online.target @9.121s
   └─network.target @9.120s
      └─NetworkManager.service @7.204s +1.914s
         └─dbus.service @7.175s
            └─basic.target @7.133s
               └─orangeipi-hardware-optimize.service @7.095s +36ms
                  └─sysinit.target @7.076s
                     └─systemd-update-utmp.service @7.027s +46ms
                        └─systemd-tmpfiles-setup.service @6.885s +78ms
                           └─systemd-journal-flush.service @6.844s +35ms
                              └─systemd-journald.service @6.583s +255ms
                                 └─orangeipi-ramlog.service @3.458s +3.119s
                                    └─var-log.mount @3.561s
                                       └─dev-zram0.device @3.276s

guard@orangezipzeroplus2h5:~$ systemd-analyze blame
6.573s logrotate.service
3.119s orangeipi-ramlog.service
2.527s networkd-dispatcher.service
2.408s man-db.service
2.074s systemd-random-seed.service
1.914s NetworkManager.service
1.907s orangeipi-zram-config.service
1.753s dev-mmio0.device
1.590s systemd-resolved.service
1.018s loadcpufreq.service
927ms wpa_supplicant.service
805ms rsyslog.service
714ms systemd-logind.service
627ms networking.service
545ms keyboard-setup.service
507ms apt-daily.service
435ms chrony.service
402ms systemd-udev-trigger.service
340ms ssh.service
317ms alsa-restore.service
304ms user@1001.service
292ms apt-daily-upgrade.service
262ms e2scrub_reap.service
255ms systemd-journald.service
196ms systemd-modules-load.service
162ms systemd-udevdev.service
146ms cpufrequtils.service
124ms hostapd.service
124ms rc-local.service
111ms dev-zram1.device
108ms dev-hugepages.mount
107ms fake-hwclock.service
106ms sysfsutils.service
104ms dev-mqueue.mount
98ms systemd-user-sessions.service
98ms sys-kernel-debug.mount
86ms systemd-sysctl.service
79ms kmod-static-nodes.service
78ms systemd-tmpfiles-setup.service
78ms systemd-sysusers.service
78ms systemd-remount-fs.service
76ms systemd-tmpfiles-setup-dev.service
73ms sysstat.service
71ms setvtrgb.service
50ms motd-news.service
46ms systemd-update-utmp.service
46ms sys-kernel-config.mount
37ms sys-fs-fuse-connections.mount
```

تصویر ۱ تحلیل زمان بوت و وضعیت اجرای موفق سرویس وب‌سرور

آزمایش ۱-۲ بررسی بازیابی خودکار در زمان Crash: برای تست پایداری و مقاومت سیستم در برابر خطا، با دستور `ps aux | grep webserver` پروسه مربوط به سرور (با شناسه PID: 1543) پیدا شد. سپس با ارسال سیگنال خاتمه اجباری (kill -9 1543)، عملکرد سیستم شبیه‌سازی شد. لاگ‌های ثبت شده در ابزار `journalctl` نشان می‌دهند که سیستم عامل بلافاصله خروج غیرمنتظره این پروسه را تشخیص داده (Main process exited, code=killed, status=9/KILL) و با توجه به پارامتر `Restart=always` در پیکربندی سرویس، آن را مجدداً زمان‌بندی و ری‌استارت کرده است. پروسه جدید با شناسه متفاوت (1554) بدون مشکل اجرا شد.

```
guard@orangeziperoplus2h5:~$ ps aux | grep webserver
guard      1543  0.1  0.7 13796 3476 ?        Ssl  14:04   0:00 /home/guard/webserver
guard      1547  0.0  0.1  5964   672 pts/0    S+   14:04   0:00 grep --color=auto webserver

guard@orangeziperoplus2h5:~$ sudo kill -9 1543
guard@orangeziperoplus2h5:~$ sudo journalctl -u webserver.service -n 20
-- Logs begin at Thu 2026-07-23 13:23:59 UTC, end at Fri 2026-07-24 14:09:19 UTC. --
Jul 23 13:24:02 orangeziperoplus2h5 systemd[1]: Started Smart Security HTTPS Web Server.
Jul 24 14:04:11 orangeziperoplus2h5 systemd[1]: webserver.service: Main process exited, code=killed, status=9/KILL
Jul 24 14:04:11 orangeziperoplus2h5 systemd[1]: webserver.service: Failed with result 'signal'.
Jul 24 14:04:14 orangeziperoplus2h5 systemd[1]: webserver.service: Scheduled restart job, restart counter is at 1.
Jul 24 14:04:14 orangeziperoplus2h5 systemd[1]: Stopped Smart Security HTTPS Web Server.
Jul 24 14:04:14 orangeziperoplus2h5 systemd[1]: Started Smart Security HTTPS Web Server.
Jul 24 14:08:53 orangeziperoplus2h5 systemd[1]: webserver.service: Main process exited, code=killed, status=9/KILL
Jul 24 14:08:53 orangeziperoplus2h5 systemd[1]: webserver.service: Failed with result 'signal'.
Jul 24 14:08:56 orangeziperoplus2h5 systemd[1]: webserver.service: Scheduled restart job, restart counter is at 2.
Jul 24 14:08:56 orangeziperoplus2h5 systemd[1]: Stopped Smart Security HTTPS Web Server.
Jul 24 14:08:56 orangeziperoplus2h5 systemd[1]: Started Smart Security HTTPS Web Server.
guard@orangeziperoplus2h5:~$ ps aux | grep webserver
guard      1554  0.0  0.7 13796 3472 ?        Ssl  14:08   0:00 /home/guard/webserver
guard      1561  0.0  0.1  5964   672 pts/0    S+   14:09   0:00 grep --color=auto webserver
```

تصویر ۲ تست توقف اجباری (kill -9) و بازیابی خودکار سرویس بر اساس لاگ‌های `journalctl`

آزمایش ۱-۳ اجرای خودکار سامانه (تست Cold Boot): برای اثبات عملکرد نهفته سیستم بدون نیاز به مداخله کاربر، برد خاموش و مجدداً به برق متصل شد. همان‌طور که در ویدیوی پیوست شده قابل مشاهده است، دستگاه بدون نیاز به اتصال کیبورد یا مانیتور (Headless) بوت شده و پس از چند ثانیه، صفحه وب سیستم در شبکه محلی در دسترس قرار گرفت که نشان از پیکربندی صحیح `systemd` و وابستگی‌های شبکه (network-online.target) دارد. فیلم این آزمایش ضمیمه گزارش شده است.

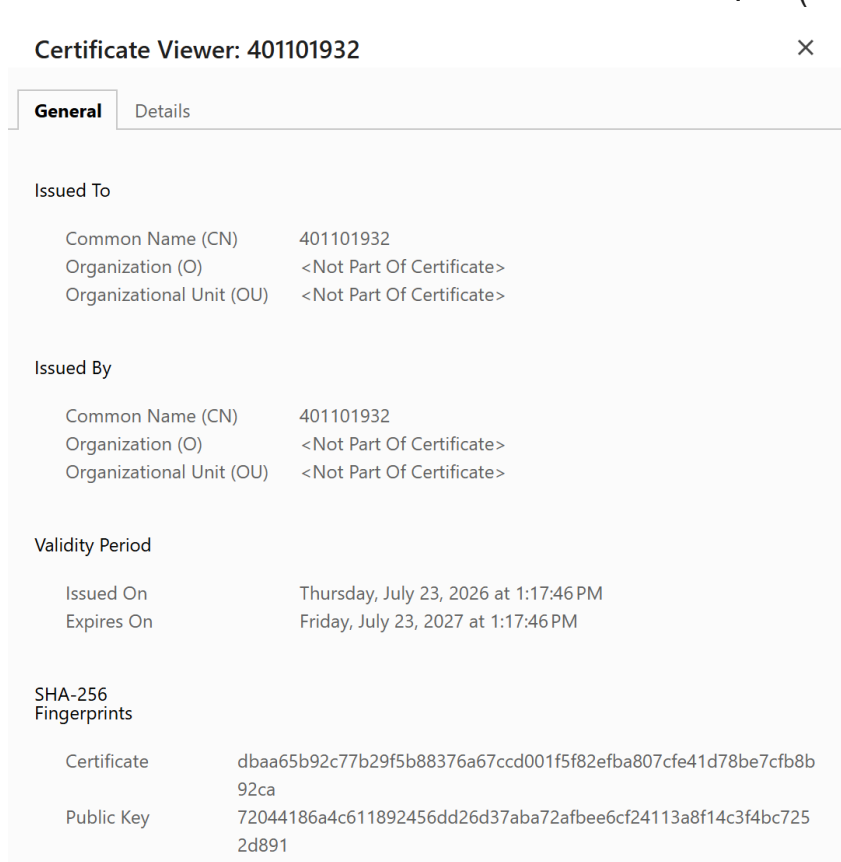
آزمایش ۱-۴ هدایت خودکار HTTP به HTTPS: برای بررسی صحت عملکرد تغییر مسیر ترافیک، آدرس سرور به صورت `http` وارد شد. بررسی با تب `Network` در قابلیت `Inspect` مرورگر و همچنین دستور `curl -I` صحت این عملکرد را تایید می‌کند و نشان می‌دهد که درخواست‌ها با کد ۳۰۱ Moved Permanently به صورت خودکار به بستر امن `HTTPS` منتقل می‌شوند.

Name	×	Headers	Preview	Response	Initiator	Timing
 10.157.161.173	▼ General					
 10.157.161.173	Request URL		http://10.157.161.173:8080/			
 data:image/png;base...	Request Method		GET			
 data:image/svg+xml;...	Status Code		 301 Moved Permanently			
	Remote Address		10.157.161.173:8080			
	Referrer Policy		strict-origin-when-cross-origin			
	▼ Response headers			☐ Raw		
	Connection			close		
	Content-Length			0		
	Location			https://10.157.161.173:8443/		
4 requests	112 B transfer					

```
C:\Users\Reza>curl -I http://10.157.161.173:8080
HTTP/1.1 301 Moved Permanently
Location: https://10.157.161.173:8443/
Content-Length: 0
Connection: close
```

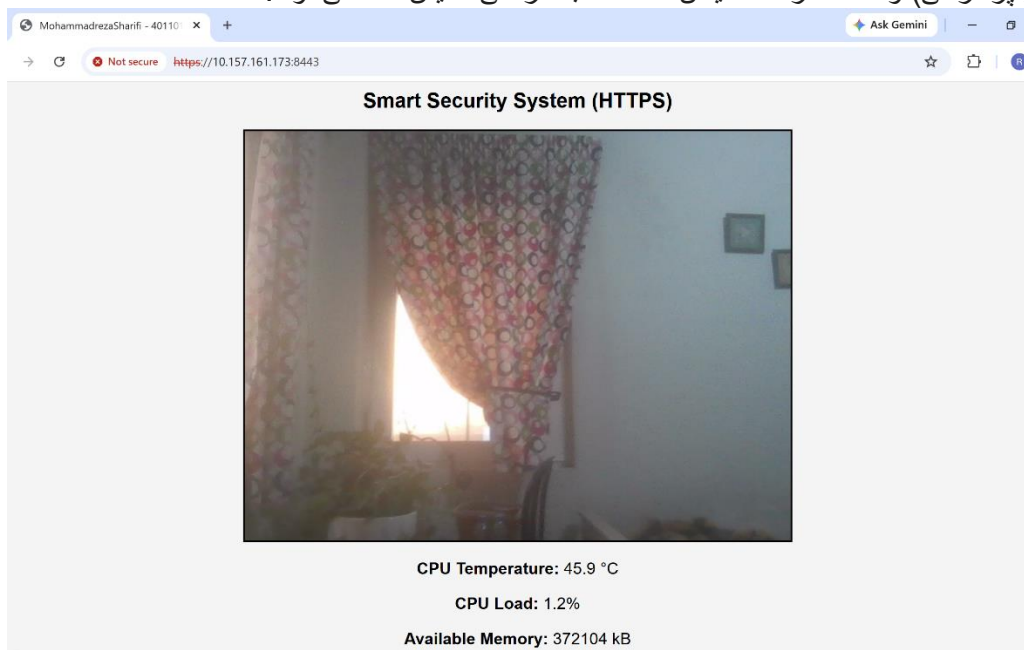
تصویر ۱ هدایت خودکار ترافیک HTTP به HTTPS (کد ۳۰۱)

آزمایش ۵-۱ بررسی گواهی Self-Signed و فیلد CN: با باز کردن تنظیمات گواهی (Certificate Viewer) در مرورگر، تایید می‌شود که گواهی به درستی صادر شده و فیلد (CN) Common Name آن دقیقاً شامل شماره دانشجویی (۴۰۱۱۰۱۹۳۲) است.



تصویر ۴ تایید تنظیم فیلد CN گواهی به شماره دانشجویی

آزمایش ۶-۱ مشاهده رابط کاربری صفحه HTML: با ورود به آدرس امن سرور، صفحه HTML بارگذاری می‌شود که در عنوان آن نام و شماره دانشجویی قرار دارد. استریم زنده دوربین به همراه اطلاعات لحظه‌ای تلمتری (دما، رم و بار پردازشی) و تعداد افراد تشخیص داده شده به درستی نمایش داده می‌شوند.



تصویر ۵ واسط کاربری وبسرور شامل استریم زنده و اطلاعات تلمتری سیستم

بخش دوم: توسعه RESTful API و پایش عملکرد سیستم

طراحی API و مستندسازی با Swagger (API Gateway)

بر اساس نیازمندی‌های پروژه، تمام منطق اصلی سرور از جمله خواندن وضعیت سنسورها (دما، حافظه و بار پردازشی)، مدیریت تاریخچه و شمارش افراد به صورت بومی با زبان C توسعه داده شده است. با این حال، جهت ارائه یک رابط کاربری استاندارد برای توسعه‌دهندگان و مستندسازی RESTful API، یک لایه واسط بسیار سبک (Thin Gateway) با استفاده از فریم‌ورک FastAPI در پایتون توسعه یافته است. این لایه واسط (API Gateway) صرفاً نقش یک Reverse Proxy را ایفا می‌کند؛ به این معنا که درخواست‌های HTTP را دریافت کرده، آن‌ها را به سرور محلی C (روی پورت ۸۴۴۳) ارسال می‌کند (Forwarding) و پاسخ خام JSON تولید شده توسط برنامه C را به کاربر بازمی‌گرداند. **قطعه کد API Gateway (لایه مستندساز پایتون):**

```
1  from fastapi import FastAPI
2  from pydantic import BaseModel
3  import requests
4  import urllib3
5
6  # Suppress insecure HTTPS warnings because we are using a self-signed cert in C
7  urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
8
9  app = FastAPI([
10     title="Smart Security System API - 401101932",
11     description="Thin Swagger documentation gateway forwarding requests to the C backend.",
12     version="1.0.0"
13 ])
14
15 # This points directly to your C server running locally on the Orange Pi
16 C_BACKEND_BASE = "https://127.0.0.1:8443"
17
18 class CommandRequest(BaseModel):
19     cmd: str
20
21 class GuardRequest(BaseModel):
22     state: int
23
24 @app.get("/api/v1/stream", tags=["Stream"])
25 def get_stream_url():
26     """Returns the redirect URL for the MJPEG video stream."""
27     res = requests.get(f"{C_BACKEND_BASE}/api/v1/stream", verify=False, allow_redirects=False)
28     return {"stream_url": res.headers.get("Location", "Stream not found")}
29
30 @app.get("/api/v1/telemetry", tags=["Telemetry"])
31 def get_telemetry():
32     """Gets CPU temperature, Memory, CPU Load, and Thermal Status from the C backend."""
33     res = requests.get(f"{C_BACKEND_BASE}/api/v1/telemetry", verify=False)
34     return res.json()
35
36 @app.get("/api/v1/persons", tags=["Detection"])
37 def get_persons():
38     """Gets the current person count."""
39     res = requests.get(f"{C_BACKEND_BASE}/api/v1/persons", verify=False)
40     return res.json()
41
42 @app.get("/api/v1/history", tags=["Detection"])
43 def get_history():
44     """Gets the last 5 detection records (In-Memory Array)."""
45     res = requests.get(f"{C_BACKEND_BASE}/api/v1/history", verify=False)
46     return res.json()
47
48 @app.post("/api/v1/command", tags=["System"])
49 def send_command(payload: CommandRequest):
50     """Sends a system command (e.g., {"cmd": "reboot"})."""
51     res = requests.post(f"{C_BACKEND_BASE}/api/v1/command", json=payload.dict(), verify=False)
52     return res.json()
53
54 # =====
55 # PART 4 ENDPOINTS ADDED
56 # =====
57
58 @app.get("/api/v1/total_detections", tags=["Blackbox"])
59 def get_total_detections():
60     """Gets the total historical person detections stored in the SQLite database."""
61     res = requests.get(f"{C_BACKEND_BASE}/api/v1/total_detections", verify=False)
62     return res.json()
63
64 @app.post("/api/v1/guard", tags=["System"])
65 def toggle_guard_mode(payload: GuardRequest):
66     """Toggles Guard Mode on (1) or off (0) to enable instant alerts and emergency MQTT topics."""
67     res = requests.post(f"{C_BACKEND_BASE}/api/v1/guard", json=payload.dict(), verify=False)
68     return res.json()
69
70 @app.post("/api/v1/heartbeat", tags=["System"])
71 def send_heartbeat():
72     """Pings the Software Watchdog to indicate the video processor is alive."""
73     res = requests.post(f"{C_BACKEND_BASE}/api/v1/heartbeat", verify=False)
74     return res.json()
```

تست زنده Endpoint ها در محیط Swagger UI

سرویس API Gateway از طریق پورت ۸۰۰۰ در دسترس است. با ورود به مسیر `docs/`، صفحه مستندات Swagger شامل عنوان و شماره دانشجویی (۴۰۱۱۰۱۹۳۲) بارگذاری می‌شود که تمامی Endpoint های درخواستی پروژه در آن تفکیک شده‌اند.

Smart Security System API - 401101932 1.0.0 OAS 3.1

Thin Swagger documentation gateway forwarding requests to the C backend.

Stream

- GET `/api/v1/stream` Get Stream Uri

Telemetry

- GET `/api/v1/telemetry` Get Telemetry

Detection

- GET `/api/v1/persons` Get Persons
- GET `/api/v1/history` Get History

System

- POST `/api/v1/command` Send Command

تصویر ۶ نمای کلی مستندات تعاملی Swagger و لیست Endpoint های توسعه یافته در سیستم

به منظور اطمینان از صحت عملکرد مسیریابی بین لایه پایتون و سرور C، تمامی Endpoint ها به صورت زنده از داخل محیط Swagger اجرا (Execute) شدند که نتایج و پاسخ های واقعی سرور (Response Body) به شرح زیر است:

۱. مسیر استریم ویدیو (GET /api/v1/stream):

با فراخوانی این مسیر، سرور آدرس محلی استریم MJPEG دوربین را جهت هدایت کلاینت بازمی‌گرداند.

GET `/api/v1/stream` Get Stream Uri

Returns the redirect URL for the MJPEG video stream.

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://10.157.161.220:8000/api/v1/stream' \
  -H 'accept: application/json'
```

Request URL

`http://10.157.161.220:8000/api/v1/stream`

Server response

Code Details

200

Response body

```
{
  "stream_url": "http://127.0.0.1:5000/video_feed"
}
```

Response headers

```
content-length: 40
content-type: application/json
date: Fri, 24 Jul 2026 21:35:37 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

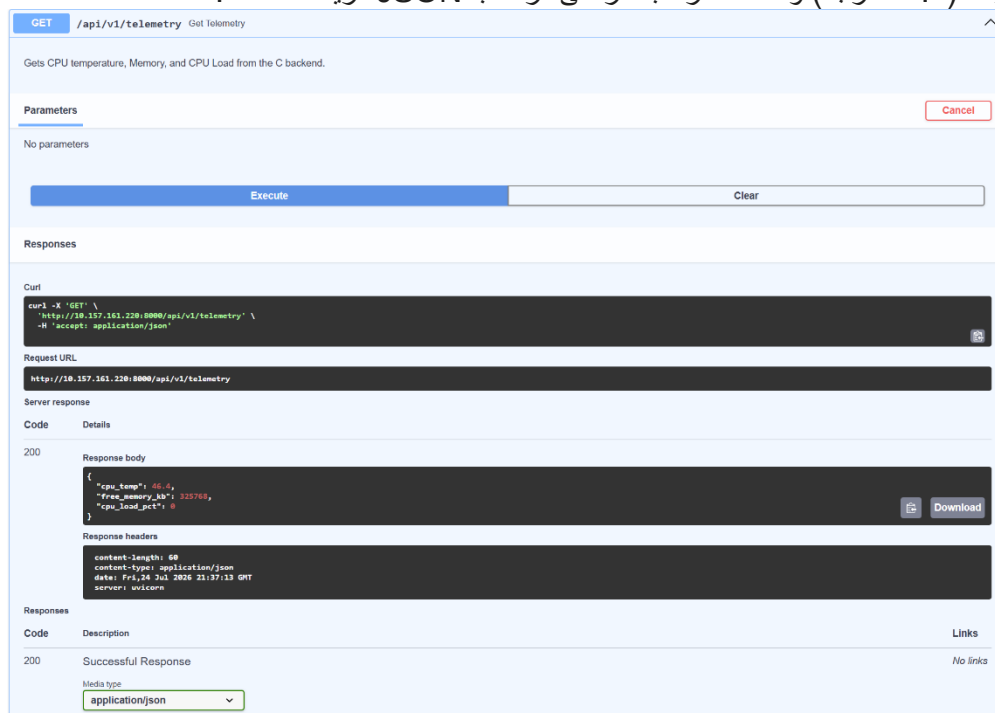
Media type

application/json

تصویر ۷ اجرای موفق مسیر stream و دریافت آدرس ویدیو اجرای موفق مسیر stream و دریافت آدرس ویدیو

۲. دریافت داده‌های تله‌متری (GET /api/v1/telemetry):

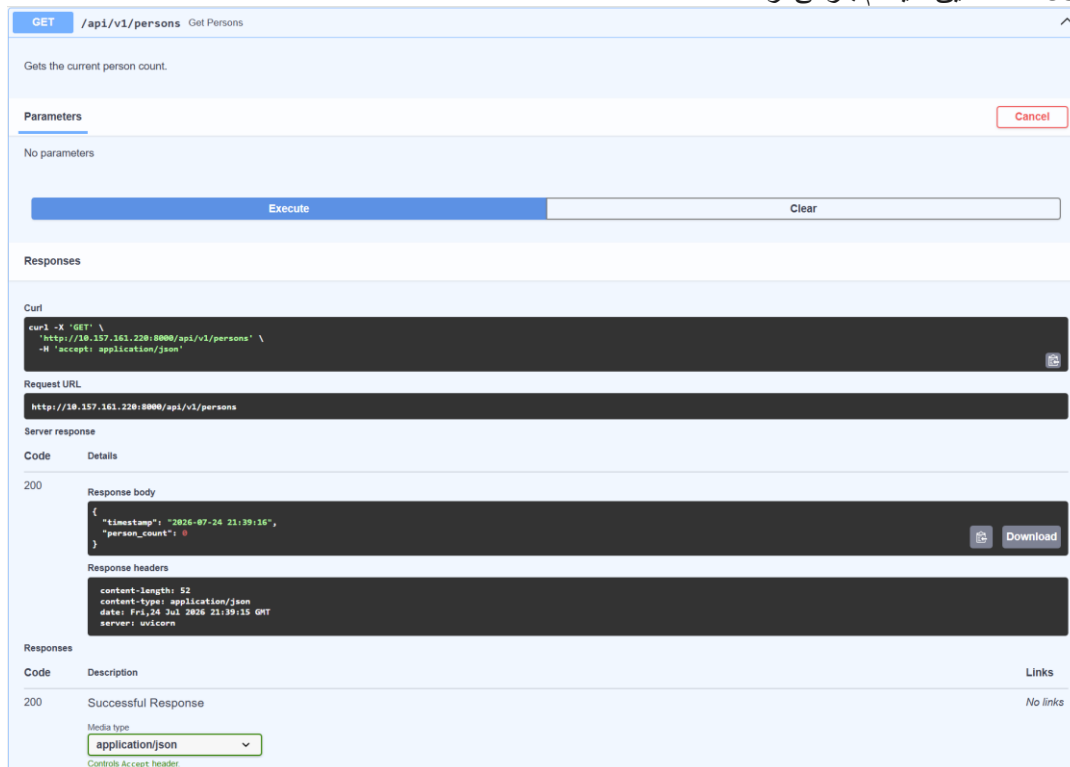
این مسیر به صورت بلادرنگ وضعیت سیستم را از کدهای C اعلام می‌کند. همان‌طور که در پاسخ مشخص است، دمای پردازنده (۴۶.۴ درجه) و حافظه آزاد به درستی در قالب JSON دریافت شده‌اند.



تصویر ۸: دریافت موفق داده‌های تله‌متری به صورت زنده

۳. وضعیت تشخیص افراد (GET /api/v1/persons):

این Endpoint تعداد افرادی که در لحظه توسط اسکریپت بینایی ماشین تشخیص داده شده‌اند را به همراه Timestamp دقیق سیستم باز می‌گرداند.



تصویر ۹: اعلام لحظه‌ای تعداد افراد تشخیص داده شده

۴. تاریخچه تشخیص (GET /api/v1/history):

با اجرای این مسیر، آرایه‌ای شامل ۵ رکورد آخر مربوط به رویدادهای تشخیص (ذخیره شده در حافظه سرور C) برگردانده می‌شود.

GET

/api/v1/history

Get History

⌵

Gets the last 5 detection records.

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://10.157.161.220:8000/api/v1/history' \
-H 'accept: application/json'
```

Request URL

```
http://10.157.161.220:8000/api/v1/history
```

Server response

Code

Details

200

Response body

```
[
  {
    "timestamp": "2026-07-24 21:16:45",
    "person_count": 0
  }
]
```

Download

Response headers

```
content-length: 56
content-type: application/json
date: Fri, 24 Jul 2026 21:40:12 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

تصویر ۱۰ دریافت تاریخچه ۵ رویداد آخر تشخیص

۵. ارسال دستورات سیستمی (POST /api/v1/command):

این مسیر برای ارسال دستورات کنترلی (مانند {"cmd": "reboot"}) طراحی شده است. برای تست پایداری و هندلینگ خطای سرور C، یک دستور نامعتبر ({"cmd": "string"}) به سرور ارسال شد. سرور C با موفقیت این خطای ورودی را تشخیص داده و پیام "error": "Unknown command" را بازگرداند که نشان‌دهنده امنیت و مدیریت خطای صحیح در لایه C است.

System

POST /api/v1/command Send Command

Sends a system command (e.g., {"cmd": "reboot"}).

Parameters

Cancel

No parameters

Request body required

application/json

Edit Value Schema

```
{  "cmd": "string"}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  'http://10.157.161.220:8080/api/v1/command' \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "cmd": "string"  }'
```

Code Details

200

Response body

```
{  "error": "Unknown command"}
```

Download

Response headers

```
content-length: 27  content-type: application/json  date: Fri, 24 Jul 2026 21:41:09 GMT  server: unicorn
```

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header Example Value Schema <pre>"string"</pre>	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string",], "msg": "string", "type": "string" }]}</pre>	No links

تصویر ۱۱ تست ارسال دستور سیستمی و مدیریت موفق دستورات نامعتبر توسط سرور C

آزمایش‌های الزامی بخش دوم

آزمایش ۱-۲ پروفایل حرارتی و پایش وضعیت پردازنده (Telemetry Profiling):

یکی از مهم‌ترین الزامات در سیستم‌های نهفته بی‌درنگ (Real-Time Embedded Systems)، پایش مداوم منابع سیستمی از جمله دمای پردازنده است. پایش از پیاده‌سازی کامل API Gateway و برای جلوگیری از آسیب‌های سخت‌افزاری ناشی از پردازش‌های سنگین (مانند اجرای مدل‌های بینایی ماشین)، رفتار حرارتی برد Orange Pi برای این منظور، اسکریپت log_temp.sh توسعه یافت. این اسکریپت وظیفه استخراج دمای پردازنده از فایل سیستمی لینوکس (/sys/class/thermal/thermal_zone0/temp/) و ثبت آن در قالب یک فایل CSV در بازه‌های زمانی ۳۰ ثانیه‌ای را بر عهده دارد: اسکریپت نوشته شده:

```
GNU nano 4.8 log_temp.sh
#!/bin/bash
echo "Time(s),Temperature(C)" > temp_log.csv
for i in {0..10}; do
    TEMP=$(cat /sys/class/thermal/thermal_zone0/temp)
    TEMP_C=$(echo "scale=1; $TEMP / 1000" | bc)
    echo "$((i * 30)), $TEMP_C" >> temp_log.csv
    echo "Sample $i: $TEMP_C °C"
    sleep 30
done
echo "Logging complete. Saved to temp_log.csv"
```

برای درک دقیق تأثیر بار پردازشی بر دمای سیستم و کالیبره کردن مکانیزم‌های کنترلی (Thermal Management)، سه سناریوی مختلف مورد آزمایش قرار گرفت:

- **حالت بیکاری سیستم (Idle State):**

در این سناریو، سرویس‌ها به صورت Background در حال اجرا هستند اما هیچ درخواست کلاینت یا پردازش سنگینی روی سیستم وجود ندارد. همان‌طور که در خروجی ثبت‌شده مشاهده می‌شود، دمای پردازنده در یک بازه امن و پایدار نوسان دارد و در نهایت لاگ‌ها در فایل temp_idle.csv ذخیره شده‌اند.

```
guard@orangepizeroplus2h5:~$ ./log_temp.sh
Sample 0: 46.4 °C
Sample 1: 48.0 °C
Sample 2: 47.4 °C
Sample 3: 45.6 °C
Sample 4: 48.0 °C
Sample 5: 49.1 °C
Sample 6: 45.8 °C
Sample 7: 45.6 °C
Sample 8: 49.1 °C
Sample 9: 49.4 °C
Sample 10: 45.3 °C
Logging complete. Saved to temp_log.csv
guard@orangepizeroplus2h5:~$ mv temp_log.csv temp_stream.csv
guard@orangepizeroplus2h5:~$ mv temp_stream.csv temp_idle.csv
```

تصویر ۱۲ حالت idle

- **حالت استریم زنده ویدیو (Video Streaming):**

با آغاز ارسال فریم‌های ویدیویی، درگیر شدن ماژول دوربین در لپ‌تاپ و هندل کردن درخواست‌های مداوم شبکه توسط سرور C، بار پردازشی افزایش می‌یابد. نتایج این تست نشان می‌دهد که دمای سیستم با یک شیب ملایم افزایش می‌یابد. این افزایش دما منطقی بوده و نشان‌دهنده فعالیت روتین شبکه است.

```
guard@orangepizeroplus2h5:~$ ./log_temp.sh
Sample 0: 48.5 °C
Sample 1: 47.5 °C
Sample 2: 50.5 °C
Sample 3: 51.2 °C
Sample 4: 48.8 °C
Sample 5: 50.0 °C
Sample 6: 52.4 °C
Sample 7: 52.2 °C
Sample 8: 50.6 °C
Sample 9: 51.3 °C
Sample 10: 53.5 °C
Logging complete. Saved to temp_log.csv
guard@orangepizeroplus2h5:~$ mv temp_log.csv temp_stream.csv
```

تصویر ۱۳ حالت video streaming

• حالت بار پردازشی سنگین استریم + تشخیص فعال:

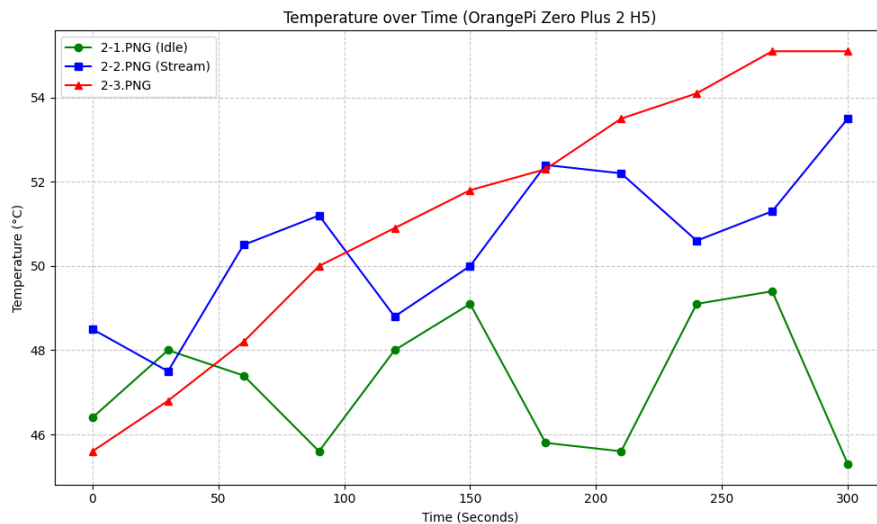
برای شبیه‌سازی شرایط بحرانی و اطمینان از عملکرد سیستم خنک‌کننده، پردازنده تحت استرس سنگین استریم + تشخیص فعال قرار گرفت در این سناریو، دما به سرعت روند صعودی به خود گرفته و به صورت پیوسته افزایش می‌یابد. این پروفایل حرارتی به ما کمک می‌کند تا آستانه بحرانی برد را شناسایی کنیم.

```
guard@orangezipzeroplus2h5:~$ ./log_temp.sh
Sample 0: 45.6 °C
Sample 1: 46.8 °C
Sample 2: 48.2 °C
Sample 3: 50.0 °C
Sample 4: 50.9 °C
Sample 5: 51.8 °C
Sample 6: 52.3 °C
Sample 7: 53.5 °C
Sample 8: 54.1 °C
Sample 9: 55.1 °C
Sample 10: 55.1 °C
Logging complete. Saved to temp_log.csv
```

تصویر ۱۴ حالت بار پردازشی سنگین استریم + تشخیص فعال

تحلیل نمودار مقایسه‌ای دما-زمان و بیشینه حرارتی

برای درک بهتر رفتار حرارتی برد Orange Pi، داده‌های ثبت شده در فایل‌های CSV مربوط به هر سه سناریو جمع‌آوری شده و در قالب یک نمودار واحد رسم شدند.



تصویر ۱۵ نمودار دما زمان سه منحنی در حالت های مختلف

همان‌طور که در نمودار فوق مشاهده می‌شود، رفتار حرارتی سیستم کاملاً تابع میزان درگیری پردازنده است:

- **منحنی سبز (Idle):** دما نوسانات طبیعی و پایداری را بین 45 تا 49.5 درجه نشان می‌دهد.
- **منحنی آبی (Stream):** با اجرای استریم تصویر، دما با یک شیب ملایم افزایش یافته و در محدوده‌ای بالاتر از حالت بیکاری (حدود 50 الی 53.5 درجه) به یک پایداری نسبی می‌رسد.
- **منحنی قرمز (Heavy Load):** شیب صعودی دما بسیار تندتر بوده و بدون نوسان کاهشی، به مرزهای بالاتری صعود می‌کند.

برای تعیین دقیق آستانه تحمل برد و تنظیم مکانیزم کنترل حرارتی در (API Thermal Throttling)، بیشینه دمای ثبت شده در طول این تست‌ها برای هر سناریو در جدول زیر خلاصه شده است:

وضعیت سیستم (سناریو)	بیشینه دمای ثبت شده (C°)
حالت بیکاری (Idle)	49.4
حالت استریم ویدیو (Streaming)	53.5

وضعیت سیستم (سناریو)	بیشینه دمای ثبت شده (C°)
حالت استرس و بار پردازشی سنگین استریم و تشخیص فعال	55.1

این اختلاف معنادار بین حالت بیکاری و بار پردازشی سنگین، نشان می‌دهد که اگرچه سیستم در دفع حرارت اولیه موفق عمل می‌کند، اما پیاده‌سازی یک مکانیزم هوشمند برای کاهش نرخ فریم (FPS) و مدیریت منابع در صورت عبور دما از مرزهای خطرناک برای تضمین پایداری سیستم کاملاً ضروری است.

آزمایش ۲-۲ بررسی مصرف حافظه (Memory Profiling):

در توسعه نرم‌افزارهای نهفته با زبان C، مدیریت دستی حافظه (Memory Management) یکی از دغدغه‌های اصلی است. در صورت عدم آزادسازی صحیح منابع (مثل رها شدن سوکت‌ها یا آزاد نشدن ساختارهای مرتبط با پروتکل SSL)، سیستم به مرور زمان دچار نشت حافظه (Memory Leak) شده و در نهایت با پر شدن ظرفیت محدود RAM برد از کار می‌افتد. برای ارزیابی این موضوع در طول زمان، اسکریپت log_mem.sh توسعه یافت.

این اسکریپت با استخراج شناسه پردازش (PID) مربوط به وب‌سرور، هر ۵ ثانیه یکبار میزان حافظه فیزیکی درگیر (RSS - Resident Set Size) توسط این پروسه را خوانده و ثبت می‌کند، اسکریپت نوشته شده:

```
#!/bin/bash
echo "Time(s),Memory(KB)" > mem_log.csv
# Get the Process ID (PID) of your C webserver
PID=$(pgrep -x webserver)

if [ -z "$PID" ]; then
    echo "Webserver is not running!"
    exit 1
fi

for i in {0..60}; do
    # Get Resident Set Size (RSS) memory of the process
    MEM=$(ps -o rss= -p $PID)
    echo "$((i * 5)), $MEM" >> mem_log.csv
    echo "Sample $i: $MEM KB"
    sleep 5
done
echo "Logging complete. Saved to mem_log.csv"
```

تحلیل خروجی وضعیت حافظه:

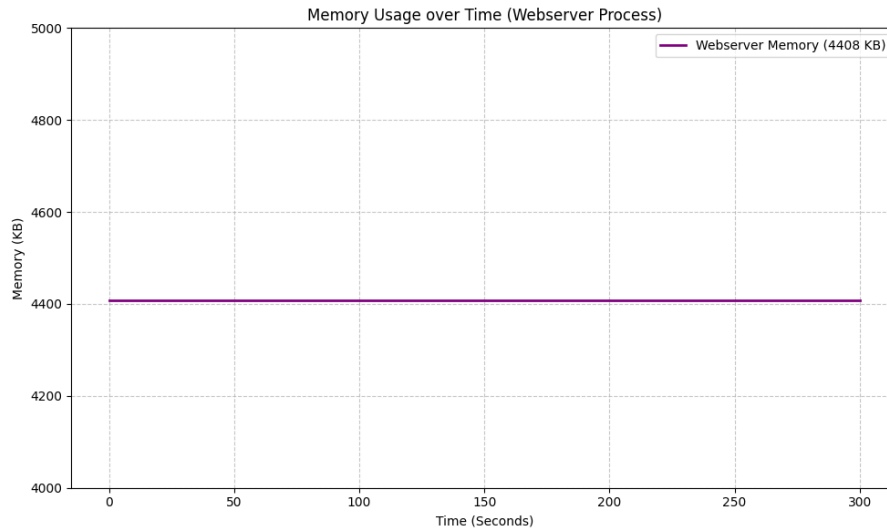
همان‌طور که در لاگ ثبت‌شده از خروجی ترمینال مشاهده می‌شود، میزان مصرف حافظه پروسه webserver به صورت کاملاً خطی و ثابت روی عدد **4408 KB** (حدود ۴.۴ مگابایت) متوقف شده است.

```
Sample 16: 4408 KB
Sample 17: 4408 KB
Sample 18: 4408 KB
Sample 19: 4408 KB
Sample 20: 4408 KB
Sample 21: 4408 KB
Sample 22: 4408 KB
Sample 23: 4408 KB
Sample 24: 4408 KB
Sample 25: 4408 KB
Sample 26: 4408 KB
Sample 27: 4408 KB
Sample 28: 4408 KB
Sample 29: 4408 KB
Sample 30: 4408 KB
Sample 31: 4408 KB
Sample 32: 4408 KB
Sample 33: 4408 KB
Sample 34: 4408 KB
Sample 35: 4408 KB
Sample 36: 4408 KB
Sample 37: 4408 KB
Sample 38: 4408 KB
Sample 39: 4408 KB
Sample 40: 4408 KB
Sample 41: 4408 KB
Sample 42: 4408 KB
Sample 43: 4408 KB
Sample 44: 4408 KB
Sample 45: 4408 KB
Sample 46: 4408 KB
Sample 47: 4408 KB
Sample 48: 4408 KB
Sample 49: 4408 KB
Sample 50: 4408 KB
Sample 51: 4408 KB
Sample 52: 4408 KB
Sample 53: 4408 KB
Sample 54: 4408 KB
Sample 55: 4408 KB
Sample 56: 4408 KB
Sample 57: 4408 KB
Sample 58: 4408 KB
Sample 59: 4408 KB
Sample 60: 4408 KB
Logging complete. Saved to mem_log.csv
```

تصویر ۱۶ ثبت کامل مصرف حافظه وب‌سرور روی 4408 KB، اثبات عدم وجود نشت حافظه (Memory Leak) طی ۶۰ بار نمونه‌برداری متوالی

این ثبات عالی طی نمونه‌برداری‌های متوالی (که در تصویر از نمونه ۱۴ تا ۶۰ قابل مشاهده است)، اثبات می‌کند که چرخه حیات درخواست‌ها در کدهای C به درستی مدیریت شده است. فراخوانی دقیق توابعی نظیر SSL_free برای

پاکسازی کانتکست رمزنگاری و `close(client_socket)` در پایان هر اتصال باعث شده تا سرور پایداری خود را حفظ کرده و هیچ‌گونه بلوکه شدن حافظه‌ای در طول اجرای متوالی سیستم رخ ندهد.



تصویر ۱۷ نمودار حافظه بر حسب زمان

همان‌طور که در نمودار فوق مشاهده می‌شود، میزان مصرف حافظه پروسه `webserver` به صورت یک خط کاملاً صاف و ثابت روی عدد **4408 KB** (حدود ۴.۴ مگابایت) متوقف شده است. این رفتار در برنامه‌نویسی سیستم‌های نهفته ایده‌آل‌ترین سناریوی ممکن است و دلایل فنی زیر را اثبات می‌کند:

- **مدیریت اصولی چرخه حیات درخواست‌ها:** فراخوانی دقیق توابعی نظیر `SSL_free` برای پاکسازی کانتکست رمزنگاری و `close(client_socket)` در پایان هر اتصال باعث شده تا هیچ‌گونه بلوکه شدن حافظه‌ای در طول اجرای متوالی سیستم رخ ندهد.
- **استفاده از بافرهای ثابت (Fixed-size Buffers):** به دلیل استفاده از تخصیص حافظه محلی با طول ثابت (مانند آرایه‌های ۲۰۴۸ بایتی در لایه سوکت)، سرور پس از رسیدن به وضعیت پایدار (Steady State) و بارگذاری اولیه گواهی‌های SSL، نیازی به درخواست حافظه تصادفی و پویا (Dynamic Allocation) از سیستم‌عامل ندارد؛ در نتیجه جای‌پای حافظه (Memory Footprint) سیستم به‌طور کامل فریز شده و پایدار می‌ماند.

آزمایش ۲-۳ تست استرس شبکه و ارزیابی تأخیر (Network Stress Test):

به منظور سنجش پایداری و تحمل‌پذیری سرور در برابر ترافیک‌های سنگین شبکه، یک سناریوی تست استرس با ایجاد درخواست‌های همزمان طراحی شد. برای این کار از اسکریپت `load_test.sh` استفاده گردید که ۵۰ حلقه تکرار شونده از درخواست‌های `curl` را به صورت همزمان (Concurrent) به سمت مسیر `api/v1/telemetry/` روانه می‌کند.

کد اسکریپت تست استرس:

```
#!/bin/bash
1 echo "Starting 50 concurrent curl loops for 30 seconds..."
2 for i in {1..50}; do
3     # -k ignores the self-signed cert warning, -s silences output
4     while true; do curl -k -s https://127.0.0.1:8443/api/v1/telemetry > /dev/null; done &
5 done
6
7
8 echo "Running... Open a second terminal and run 'htop' or 'top' to observe CPU/RAM."
9 sleep 30
10
11 echo "Stopping all curl requests..."
12 killall curl
13 echo "Test complete"
```

تحلیل مشاهدات و رفتار سیستم در حین تست

۱. وضعیت پایه (Baseline):

پیش از آغاز تست استرس، سیستم در وضعیت پایداری قرار داشت. دمای پردازنده حدود ۵۳.۰ درجه سانتی‌گراد، بار پردازشی (CPU Load) معادل ۱.۲٪ و حافظه آزاد رم در محدوده امن ۳۲۸ مگابایت بود.

CPU Temp: 53.0 °C | CPU Load: 1.2% | Free RAM: 328324 kB

Persons Detected: 0

تصویر ۱۸ وضعیت پایدار سیستم پیش از شروع تست استرس

۲. وضعیت منابع تحت فشار (Resource Exhaustion):

با اجرای اسکریپت و سرازیر شدن سیل درخواست‌ها، تغییرات چشمگیری در منابع سیستم رخ داد:

- افت شدید حافظه آزاد:** حافظه RAM آزاد سیستم با افت شدیدی مواجه شده و به ۱۶۹ مگابایت کاهش یافت. با بررسی خروجی ابزار top متوجه می‌شویم که این افت حافظه لزوماً به خاطر نشت حافظه در سرور C نیست، بلکه به این دلیل است که اسکریپت تست روی خود برد اجرا شده و سیستم‌عامل دها پروسه مجزای curl را در پس‌زمینه ایجاد کرده است که هر کدام بخشی از حافظه را اشغال کرده‌اند.
- افزایش دما و بار پردازشی:** بار پردازشی (Load) به ۱۱.۱٪ جهش کرد و دمای CPU به ۵۶.۶ درجه سانتی‌گراد رسید که نشان‌دهنده درگیری شدید هسته‌های پردازشی برای هندل کردن سوکت‌های شبکه و عملیات رمزنگاری SSL است.

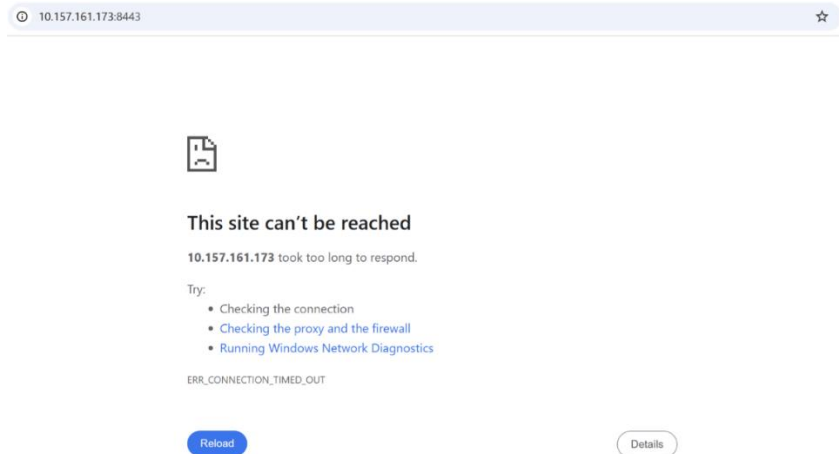
top - 20:33:10 up 11 min, 2 users, load average: 0.21, 0.09, 0.02									
Tasks: 209 total, 1 running, 208 sleeping, 0 stopped, 0 zombie									
%Cpu(s): 11.8 us, 2.5 sy, 0.0 ni, 85.6 id, 0.0 wa, 0.0 hi, 0.1 si, 0.0 st									
MiB Mem : 477.6 total, 113.5 free, 233.0 used, 131.2 buff/cache									
MiB Swap: 238.8 total, 238.3 free, 0.5 used, 230.8 avail Mem									
PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM
974	guard	20	0	13916	4420	3828	S	7.3	0.9
1781	guard	20	0	26084	10548	8580	S	2.6	2.2
1779	guard	20	0	26084	10856	8888	S	2.3	2.2
1780	guard	20	0	26084	10772	8676	S	2.3	2.2
1782	guard	20	0	26084	10608	8644	S	2.3	2.2
1783	guard	20	0	26084	10716	8752	S	2.3	2.2
1784	guard	20	0	26084	10884	8916	S	2.3	2.2
1766	guard	20	0	26084	10892	8792	S	1.7	2.2
1774	guard	20	0	25156	9108	7864	S	1.0	1.9
460	root	-2	0	0	0	0	S	0.7	0.0
979	guard	20	0	57984	48956	11060	S	0.7	10.0
1615	guard	20	0	8912	3268	2608	R	0.7	0.7
1775	guard	20	0	25156	9176	7932	S	0.7	1.9
10	root	20	0	0	0	0	I	0.3	0.0
553	root	19	-1	37968	7232	6140	S	0.3	1.5
1603	guard	20	0	14660	4660	3508	S	0.3	1.0
1627	guard	20	0	6464	1588	1336	S	0.3	0.3
1639	guard	20	0	25156	9180	7932	S	0.3	1.9
1	root	20	0	166700	9932	7356	S	0.0	2.0
2	root	20	0	0	0	0	S	0.0	0.0
3	root	0	-20	0	0	0	I	0.0	0.0
4	root	0	-20	0	0	0	I	0.0	0.0
6	root	0	-20	0	0	0	I	0.0	0.0
7	root	20	0	0	0	0	I	0.0	0.0
8	root	0	-20	0	0	0	I	0.0	0.0
9	root	20	0	0	0	0	S	0.0	0.0
11	root	rt	0	0	0	0	S	0.0	0.0
12	root	20	0	0	0	0	S	0.0	0.0
13	root	20	0	0	0	0	S	0.0	0.0
14	root	rt	0	0	0	0	S	0.0	0.0
15	root	20	0	0	0	0	S	0.0	0.0
17	root	0	-20	0	0	0	I	0.0	0.0
18	root	20	0	0	0	0	S	0.0	0.0
19	root	rt	0	0	0	0	S	0.0	0.0
20	root	20	0	0	0	0	S	0.0	0.0
23	root	20	0	0	0	0	S	0.0	0.0
24	root	rt	0	0	0	0	S	0.0	0.0
25	root	20	0	0	0	0	S	0.0	0.0
26	root	20	0	0	0	0	I	0.0	0.0
28	root	20	0	0	0	0	S	0.0	0.0
29	root	0	-20	0	0	0	I	0.0	0.0
30	root	20	0	0	0	0	S	0.0	0.0

CPU Temp: 56.6 °C | CPU Load: 11.1% | Free RAM: 169524 kB

Persons Detected: 0

تصویر ۱۹ افت شدید حافظه و اشغال منابع توسط پروسه‌های متعدد curl در خروجی top

۳. تأخیر و از دسترس خارج شدن سرور (Timeout & Latency):
مهم‌ترین خروجی این تست، بررسی میزان پاسخ‌گویی سرور بود. در حین اجرای این ۵۰ درخواست همزمان و بی‌وقفه، تلاش برای باز کردن رابط کاربری وب سرور از طریق مرورگر شبکه محلی با خطای ERR_CONNECTION_TIMED_OUT مواجه شد.

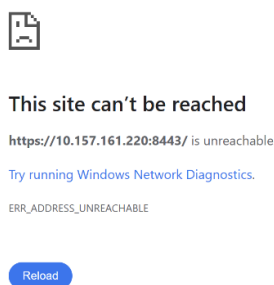


تصویر ۲۰ از دسترس خارج شدن سرور و بروز خطای Timeout تحت بار ترافیکی سنگین

نتیجه‌گیری و بازیابی خودکار (System Recovery):
وب‌سرور توسعه‌یافته با زبان C، به دلیل ماهیت تک‌نخی (Single-threaded) در حلقه accept، در مواجهه با ترافیک شدید و ده‌ها Handshake همزمان، دچار پر شدن صف اتصالات (Listen Backlog) شده و به طور موقت از دسترس خارج می‌شود (Timeout). با این حال، سیستم به گونه‌ای طراحی شده است که دارای خاصیت تاب‌آوری (Resilience) بالایی است. به محض پایان یافتن ترافیک غیرعادی (اتمام اسکریپت پس از ۳۰ ثانیه)، سرور بلافاصله صف شبکه را تخلیه کرده و بدون نیاز به مداخله دستی، مجدداً پاسخ‌گویی به کلاینت‌ها را از سر می‌گیرد. علاوه بر این، در صورتی که این اختلال باعث قطع ارتباط دوربین برای بیش از ۳۰ ثانیه شود، مکانیزم Watchdog نرم‌افزاری تعبیه‌شده در سیستم وارد عمل شده و سرویس را به طور خودکار ری‌استارت می‌کند تا پایداری سامانه در شرایط بحرانی تضمین گردد.
این رفتار در معماری سیستم‌های نهفته کاملاً مورد انتظار است. در واقع، این محدودیت در هندلینگ ترافیک عظیم، بهای (Trade-off) معماری بسیار سبکوزن (Lightweight) این سرور است که باعث شده سیستم بتواند با اشغال تنها ۴ مگابایت رم، عملکرد اصلی خود یعنی پایش امنیتی را با موفقیت انجام دهد.

آزمایش ۲-۴ بررسی پایداری و تاب‌آوری سامانه در برابر قطع شبکه (Network Drop Test):
۱. رفتار سیستم حین قطعی شبکه:

بلافاصله پس از قطع شدن ارتباط، کلاینت (مرورگر) توانایی دریافت فریم‌های ویدیویی و به‌روزرسانی اطلاعات سیستم را از دست داد و با خطای ERR_ADDRESS_UNREACHABLE مواجه شد.



تصویر ۲۱ از دسترس خارج شدن رابط کاربری و بروز خطای ارتباطی در مرورگر به دلیل قطع شبکه

۲. تحلیل وضعیت سرور C و لاگ‌های سیستم:

مهم‌ترین بخش این آزمایش، بررسی رفتار هسته اصلی سامانه (وب‌سرور C) در زمان قطعی است. در سیستم‌های نهفته با برنامه‌نویسی غیرتطبیقی، قطع ناگهانی سوکت کلاینت می‌تواند منجر به پرتاب خطای سیستمی (مانند SIGPIPE) و Crash شدن کل پروسه سرور شود. اما با بررسی لاگ‌های سرویس از طریق ابزار journalctl مشاهده می‌شود که سیستم کاملاً پایدار مانده است.

```
guard@orangeziperplus2h5:~$ sudo journalctl -u webserver.service -n 50
-- Logs begin at Fri 2026-07-24 21:16:42 UTC, end at Fri 2026-07-24 21:30:41 UTC. --
Jul 24 21:16:45 orangeziperplus2h5 systemd[1]: Started Smart Security HTTPS Web Server.
guard@orangeziperplus2h5:~$ sudo journalctl -u apigateway.service -n 50
-- Logs begin at Fri 2026-07-24 21:16:42 UTC, end at Fri 2026-07-24 21:30:51 UTC. --
Jul 24 21:16:45 orangeziperplus2h5 systemd[1]: Started FastAPI Swagger Gateway for Smart Security.
Jul 24 21:16:52 orangeziperplus2h5 python3[955]: INFO: Started server process [955]
Jul 24 21:16:52 orangeziperplus2h5 python3[955]: INFO: Waiting for application startup.
Jul 24 21:16:52 orangeziperplus2h5 python3[955]: INFO: Application startup complete.
Jul 24 21:16:52 orangeziperplus2h5 python3[955]: INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

تصویر ۲۲ لاگ‌های سرویس‌های webserver و apigateway که نشان‌دهنده عدم Crash و اجرای پیوسته آن‌هاست

همان‌طور که در لاگ‌ها مشخص است، هیچ‌گونه خروج غیرمنتظره یا تلاش مجدد برای اجرای سرویس (Restart) ثبت نشده است. دلیل این پایداری، مدیریت صحیح سوکت‌ها در سمت سرور C است. هنگامی که شبکه قطع می‌شود، توابع SSL_read و SSL_write با خطا مواجه می‌شوند. کدی که برای سرور نوشته شده، این خطا را هندل کرده، سوکت مربوط به کلاینت قطع‌شده را می‌بندد (SSL_free و close) و به سادگی در حلقه while (۱) روی تابع accept منتظر اتصالات جدید می‌ماند.

۳. بازیابی خودکار و اتصال مجدد (Recovery):

پس از گذشت ۲ دقیقه، شبکه مجدداً متصل شد. به دلیل پایداری سرور در مرحله قبل، هیچ نیازی به لاگین کردن به برد (SSH) و یا ری‌استارت کردن دستی سرویس‌ها نبود. تنها با یک بار بارگذاری مجدد (Reload) صفحه وب، سامانه فوراً به حالت عملیاتی بازگشت. استریم زنده مجدداً برقرار شد و داده‌های تله‌متری (مانند دمای ۴۲.۶ درجه) به درستی در صفحه نمایش داده شدند.

Not secure https://10.157.161.220:8443

Smart Security System (HTTPS)



CPU Temp: 42.6 °C | CPU Load: 1.2% | Free RAM: 327424 kB

Persons Detected: 0

تصویر ۲۳ بازیابی موفقیت‌آمیز استریم و اطلاعات تله‌متری بلافاصله پس از اتصال مجدد شبکه

این آزمایش اثبات می‌کند که معماری شبکه سامانه کاملاً در برابر اختلالات ارتباطی مقاوم (Fault-Tolerant) بوده و می‌تواند بدون نیاز به مداخله انسانی، به سرویس‌دهی خود ادامه دهد.

بخش سوم: پردازش تصویر، ایمیل و MQTT

الف) پردازش تصویر و تشخیص انسان

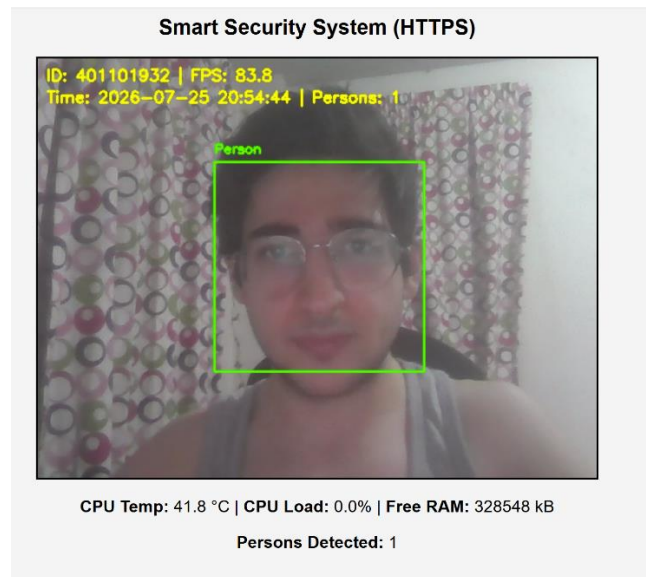
برای پیاده‌سازی بخش بینایی ماشین و شمارش افراد، از الگوریتم سبک و سریع Haar Cascade (مدل haarcascade_frontalface_default.xml) استفاده شده است. از آنجا که این الگوریتم بر پایه پردازش لبه‌ها و اختلاف تضاد نوری (Contrast) در نواحی مختلف چهره (مانند تیرگی چشم‌ها نسبت به گونه‌ها) کار می‌کند، شرایط نوری محیط تأثیر مستقیمی بر دقت و عملکرد آن دارد.

آزمایش ۱-۳: سنجش دقت تشخیص در ۴ وضعیت نوری:

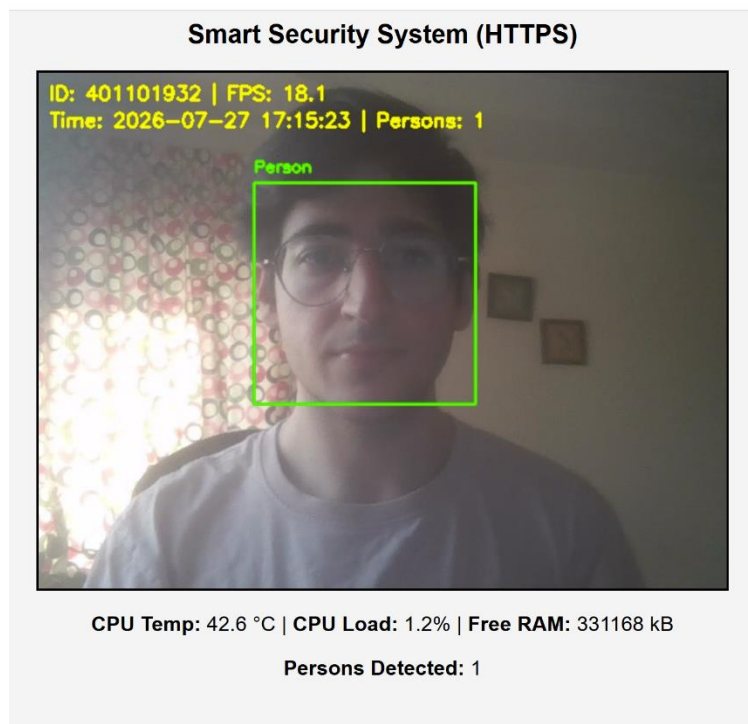
برای ارزیابی پایداری سیستم، عملکرد الگوریتم تشخیص چهره در ۴ سناریوی نوری مختلف مورد آزمایش قرار گرفت.

۱. نور روز و نور مصنوعی (شرایط ایده‌آل):

در این حالت به دلیل وجود تضاد نوری مناسب و واضح بودن لبه‌های چهره، سیستم با بالاترین دقت ممکن عمل می‌کند.



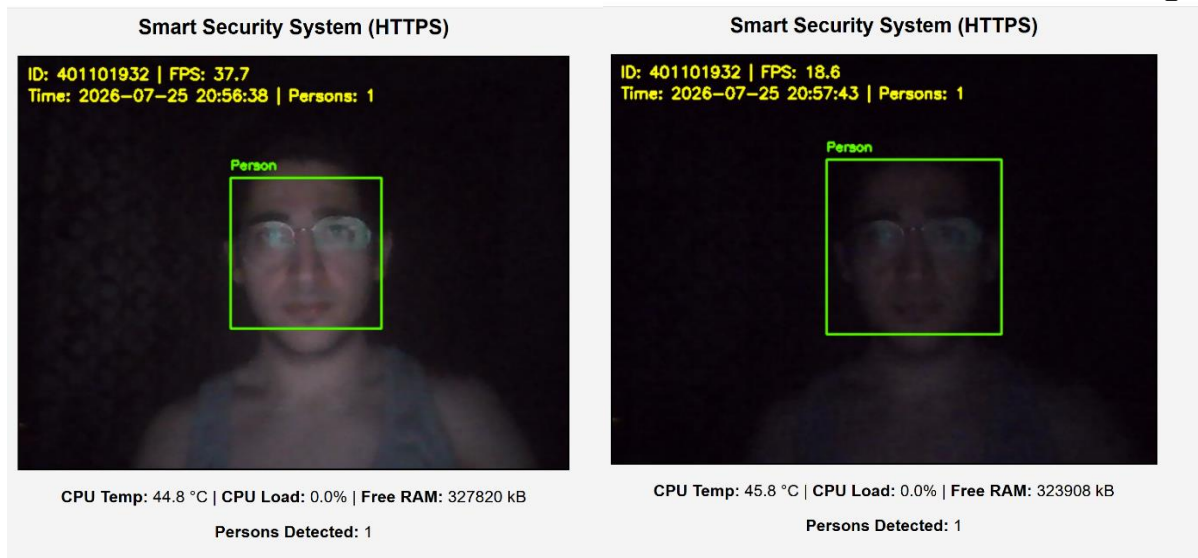
تصویر ۲۴: تشخیص چهره در نور مصنوعی (لامپ)



تصویر ۲۵: تشخیص چهره در نور طبیعی روز

۲. محیط تاریک و نور کم (Low Light):

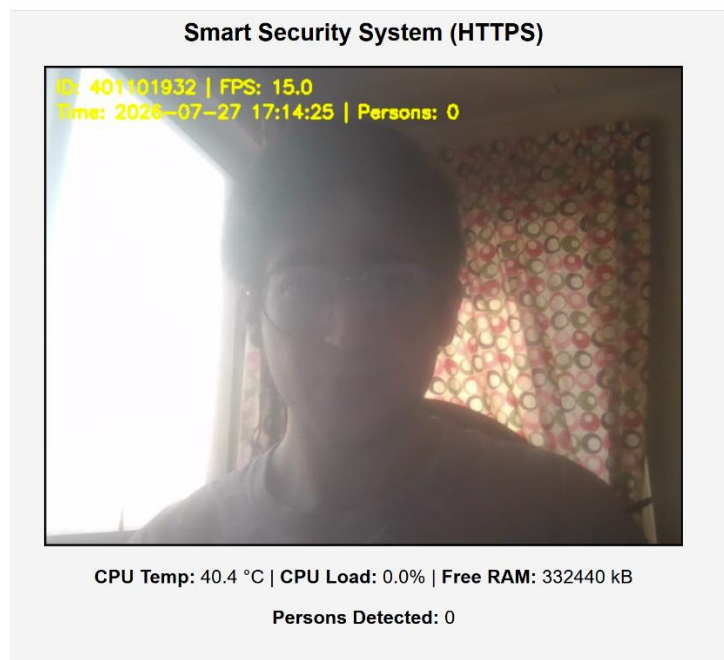
در محیط‌های کم‌نور، نویز تصویر افزایش یافته و کنتراست چهره به شدت افت می‌کند. مشاهدات نشان داد که الگوریتم در اکثر فریم‌ها موفق به تشخیص می‌شود اما در برخی چهره را گم می‌کند اما عملکرد کلی خوب و قابل دفاع است.



تصویر ۲۶ تشخیص چهره در محیط تاریک و محیط کم نور

۳. نور از پشت (Backlight):

این سناریو چالش‌برانگیزترین حالت برای الگوریتم‌های مبتنی بر روشنایی است. وقتی منبع نور (مانند پنجره) پشت سر فرد قرار دارد، چهره به حالت سایه‌روشن (Silhouette) درآمده و جزئیات چشم و بینی از بین می‌رود. در این حالت سیستم در اکثر مواقع قادر به تشخیص نیست و تنها با تغییر زاویه جزئی صورت، چهره برای لحظاتی شناسایی می‌شود.



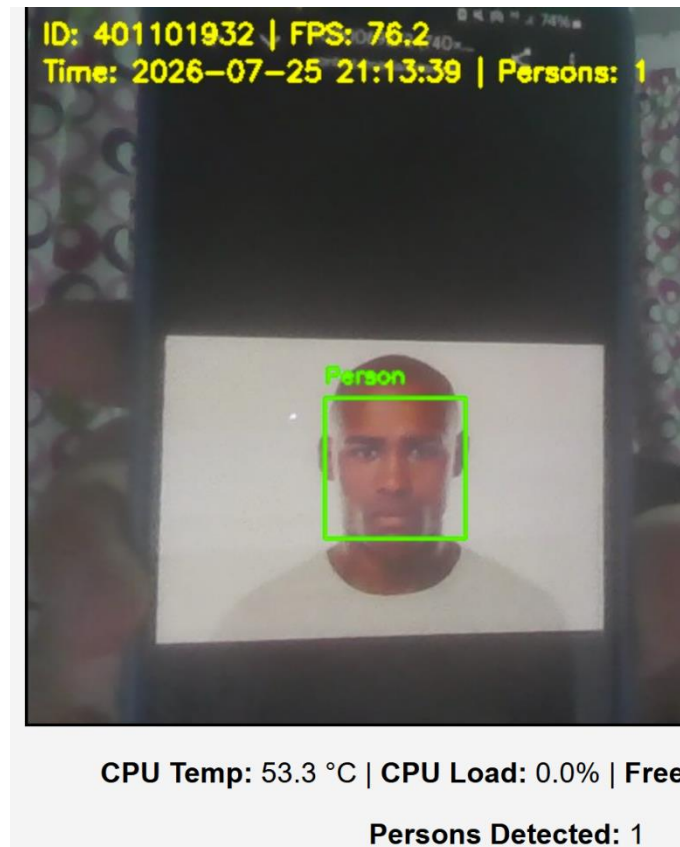
تصویر ۲۷ تشخیص چهره در حضور منبع نوری از پشت

جدول خلاصه عملکرد تشخیص در شرایط نوری مختلف:

وضعیت نوری محیط	نرخ دقت تقریبی	تحلیل عملکرد
نور روز (Daylight)	۹۰٪	ایده‌آل، لبه‌های چهره کاملاً واضح است.
نور مصنوعی (لامپ)	۹۰٪	بسیار مطلوب، بدون مشکل خاص.
نور کم (تاریکی)	۷۰٪	عملکرد نسبتاً قابل قبول با در نظر گرفتن شرایط نوری
نور از پشت (Backlight)	۴۰٪	ضعیف‌ترین عملکرد، تاریک شدن چهره به دلیل منبع نور مزاحم.

آزمایش ۲-۳ تست فریب سیستم (Spoofing):

یکی از چالش‌های امنیتی در سیستم‌های پردازش تصویر ساده، عدم توانایی در درک عمق و زنده بودن سوژه است. برای تست این آسیب‌پذیری، یک عکس چاپ شده/دیجیتالی از چهره یک انسان روی صفحه نمایش موبایل در برابر دوربین قرار گرفت.



تصویر ۲۸ سیستم فریب خورده از روی عکس موبایل

تحلیل و مشاهدات:

همان‌طور که در خروجی مشخص است، سیستم فریب خورده و عکس روی موبایل را به عنوان یک "فرد" (Person: 1) با موفقیت تشخیص داد و ایمیل هشدار را ارسال کرد. دلیل این امر آن است که مدل‌های آشنایی

(Haar Cascades) تنها به دنبال الگوهای پیکسلی دوبعدی (مستطیل‌های تیره و روشن) می‌گردند و هیچ درکی از حجم سه بعدی یا علائم حیاتی سوژه ندارند.

پیشنهاد راهکارها برای جلوگیری از فریب سیستم:

برای ارتقاء امنیت و جلوگیری از حملات Presentation Attack (مانند نمایش عکس یا ماسک)، می‌توان از راهکارهای زیر استفاده کرد:

۱. **تشخیص زنده بودن (Liveness Detection - نرم‌افزاری):** اضافه کردن یک لایه پردازشی برای بررسی علائم حیاتی مانند تشخیص پلک زدن (Blink Detection) یا درخواست از سوژه برای حرکت دادن سر.

۲. **بررسی بافت و بازتاب (Texture Analysis):** استفاده از شبکه‌های عصبی عمیق (Deep Learning) که قادرند تفاوت بافت پوست واقعی انسان را با بازتاب نور (Glare) از روی صفحه نمایش موبایل یا کاغذ تشخیص دهند.

۳. **سنسورهای سخت‌افزاری (Depth/IR Cameras):** استفاده از دوربین‌های دارای سنسور عمق (مانند سنسورهای Time of Flight یا Kinect) و دوربین‌های مادون قرمز که می‌توانند حجم سه‌بعدی چهره و حرارت را اسکن کنند و به هیچ‌وجه با عکس‌های دوبعدی فریب نمی‌خورند.

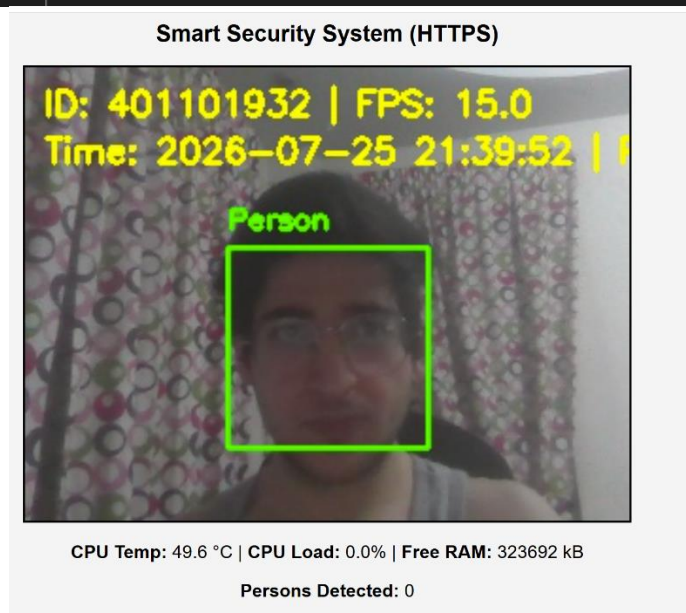
آزمایش ۳-۳ بررسی تأثیر رزولوشن تصویر بر عملکرد سیستم (تست بار پردازشی):

برای یافتن نقطه بهینه (Trade-off) میان کیفیت تصویر و پایداری سیستم، رزولوشن استریم دوربین در کد `laptop_streamer.py` در سه سطح مختلف (پایین، متوسط و بالا) تنظیم شد و پس از ۵ دقیقه اجرای مداوم در هر حالت، پارامترهای حیاتی سیستم اندازه‌گیری شدند.

۱. بررسی در رزولوشن پایین (320x240)

در این حالت، ابعاد تصویر به حداقل رسانده شد تا رفتار سیستم در سبک‌ترین حالت پردازشی ارزیابی شود.

```
# Resize for output stream
# CHANGE THESE TWO NUMBERS FOR EACH TEST:
TEST_WIDTH = 320
TEST_HEIGHT = 240
```



تصویر ۲۹ خروجی سیستم در رزولوشن پایین. دمای ۴۹.۶ درجه و نرخ فریم ۱۵.۰

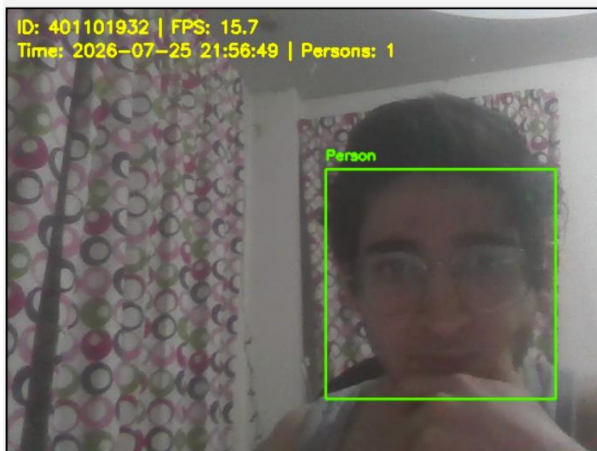
مشاهدات: به خاطر افت کیفیت بصری تصویر، در این حالت سیستم به راحتی نمی‌تواند چهره را با دقت بالا تشخیص دهد. اگر چه که سرعت در این حالت بالاترین است اما به خاطر پایین آمدن کیفیت تصویر جزئیات تصویر نیز کمتر میشود و مدل دچار مشکل میشود. دما در محدوده امن ۴۹.۶ درجه سانتی‌گراد پایدار ماند و نرخ فریم (FPS) روی عدد ۱۵.۰ قرار گرفت.

۲. بررسی در رزولوشن متوسط (640x480)

این حالت به عنوان رزولوشن استاندارد و پیش‌فرض سیستم در نظر گرفته شده است.

```
# Resize for output stream
# CHANGE THESE TWO NUMBERS FOR EACH TEST:
TEST_WIDTH = 640
TEST_HEIGHT = 480
```

Smart Security System (HTTPS)



CPU Temp: 50.5 °C | CPU Load: 0.0% | Free RAM: 318828 kB

Persons Detected: 1

تصویر ۳۰ خروجی سیستم در رزولوشن متوسط. دمای ۵۰.۵ درجه و نرخ فریم ۱۵.۷

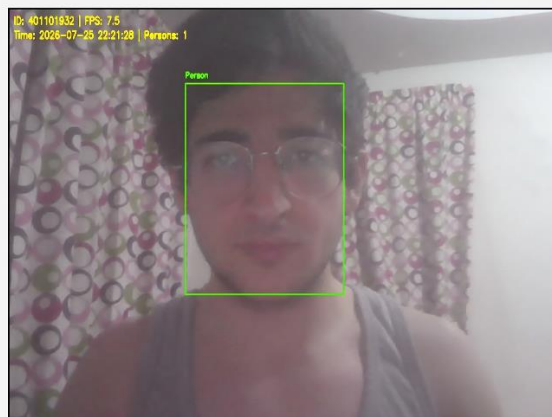
مشاهدات: کیفیت تصویر برای مانیتورینگ امنیتی بسیار مطلوب است و همین باعث شد که با دقت نسبتاً بالایی چهره را مدل تشخیص بدهد. دمای پردازنده با یک افزایش منطقی به ۵۰.۵ درجه رسید. نکته جالب توجه حفظ و حتی بهبود جزئی نرخ فریم (۱۵.۷) بود که نشان می‌دهد پهنای باند شبکه و قدرت پردازشی برد در این رزولوشن به خوبی با یکدیگر همگام هستند.

۳. بررسی در رزولوشن بالا (1280x720)

در این سناریو، ابعاد تصویر به شدت افزایش یافت تا سیستم تحت فشار پردازشی (Stress Test) قرار گیرد.

```
# Resize for output stream
CHANGE THESE TWO NUMBERS FOR EACH TEST:
TEST_WIDTH = 1280
TEST_HEIGHT = 720
```

Smart Security System (HTTPS)



CPU Temp: 57.5 °C | CPU Load: 0.0% | Free RAM: 318528 kB

تصویر ۳۱ خروجی سیستم در رزولوشن بالا. افت شدید FPS به ۷.۵ و افزایش دما به ۵۷.۵ درجه

مشاهدات: افزایش پیکسل‌ها باعث شد تا بار پردازشی روی الگوریتم تشخیص و همچنین عملیات فشرده‌سازی JPEG به شدت بالا برود. در نتیجه، نرخ فریم به بیش از نصف کاهش یافت (۷.۵ فریم بر ثانیه) و دمای پردازنده با یک جهش قابل توجه به ۵۷.۵ درجه سانتی‌گراد رسید که برای کارکرد طولانی‌مدت سیستم می‌تواند نگران‌کننده باشد.

اما به دلیل این که بهترین کیفیت در این حالت میباشد، دقت مدل در این حالت بسیار مطلوب بود اما هم با افزایش دما و هم کاهش سرعت همراه بود.

خلاصه نتایج استخراج شده از این سه آزمایش در جدول زیر آورده شده است:

ابعاد تصویر (Resolution)	نرخ فریم (FPS)	دمای پردازنده (پس از ۵ دقیقه)	حافظه آزاد سیستم (Free RAM)	دقت تشخیص چهره
320x240 (پایین)	۱۵.۰	C° ۴۹.۶	kB ۳۲۳,۶۹۲	تشخیص نوسانی
640x480 (متوسط)	۱۵.۷	C° ۵۰.۵	kB ۳۱۸,۸۲۸	مطلوب/بسیار مطلوب
1280x720 (بالا)	۷.۵	C° ۵۷.۵	kB ۳۱۸,۵۲۸	بسیار مطلوب

تحلیل نقطه بهینه:

با بررسی جدول بالا مشخص می‌شود که رزولوشن 640x480 نقطه بهینه (Sweet Spot) این سیستم است. در رزولوشن 1280x720، اگرچه کیفیت تصویر بالاتر است، اما تأخیر در پردازش هر فریم باعث نصف شدن FPS و داغ شدن شدید پردازنده (۵۷.۵ درجه) می‌شود که در یک سیستم نهفته به هیچ وجه توجیه‌پذیر نیست. از سوی دیگر، رزولوشن 320x240 تفاوت دمایی بسیار کمی (حدود ۱ درجه) با 640x480 دارد اما جزئیات تصویر در آن به اندازه کافی نیست، بنابراین عملکرد مدل دچار مشکل می‌شود.

آزمایش ۳-۴ بررسی عملکرد پیام‌رسان (LWT) در پروتکل MQTT:

در سیستم‌های اینترنت اشیا (IoT) و نهفته، همواره احتمال قطعی ناگهانی ارتباط به دلایلی نظیر قطع برق برد، کرش کردن نرم‌افزار یا اختلال فیزیکی در شبکه وجود دارد. در چنین شرایطی، کلاینت فرصت نمی‌کند پیام خروج استاندارد (Disconnect) به Broker ارسال کند. برای مدیریت این بحران، از ویژگی LWT در پروتکل MQTT استفاده شد.

در کدهای نوشته شده به زبان C، هنگام پیکربندی اولیه MQTT، پیام وصیت‌نامه با استفاده از تابع `mosquitto_will_set` به صورت زیر تنظیم شده است:

```
147 mosquitto_will_set(mosq, "telemetry/" STUDENT_ID "/home", strlen("{\"status\":\"OFFLINE\"}"), "{\"status\":\"OFFLINE\"}", 1, true);
148
```

این خط به Broker (نرم‌افزار Mosquitto در کامپیوتر شخصی) اعلام می‌کند که اگر ارتباط برد Orange Pi به صورت غیرمنتظره قطع شد، پیام `{"status":"OFFLINE"}` را به طور خودکار در تاپیک تله‌متری منتشر کند. مراحل و مشاهدات تست قطعی ناگهانی:

برای ارزیابی این قابلیت، ابتدا ارتباط زنده مانیتور شد و سپس پروسه وب‌سرور به صورت اجباری متوقف گردید.

```
guard@orangeziperoplus2h5:~$ sudo killall -9 webserver
guard@orangeziperoplus2h5:~$
```

تصویر ۳۲ شبیه‌سازی کرش کردن سیستم با ارسال سیگنال خاتمه اجباری `kill -9` به پروسه وب‌سرور

با اجرای دستور `sudo killall -9 webserver`، پروسه پردازشی روی برد فوراً متوقف شد. از آنجا که اتصال سوکت MQTT به صورت ناگهانی بسته شد، Broker بلافاصله وارد عمل گردید.

```
C:\Program Files\Mosquitto>mosquitto_sub -h localhost -t "#" -u guard_user -P secure_password123 -v
telemetry/401101932/home {"status":"OFFLINE"}
persons/401101932/home {"student_id":"401101932", "count":1, "timestamp":"2026-07-25 19:27:52"}
telemetry/401101932/home {"student_id":"401101932", "cpu_temp":42.1, "timestamp":"2026-07-25 19:27:52"}
persons/401101932/home {"student_id":"401101932", "count":0, "timestamp":"2026-07-25 19:27:53"}
telemetry/401101932/home {"student_id":"401101932", "cpu_temp":42.2, "timestamp":"2026-07-25 19:27:53"}
telemetry/401101932/home {"status":"OFFLINE"}
persons/401101932/home {"student_id":"401101932", "count":1, "timestamp":"2026-07-25 19:28:05"}
telemetry/401101932/home {"student_id":"401101932", "cpu_temp":42.9, "timestamp":"2026-07-25 19:28:05"}
persons/401101932/home {"student_id":"401101932", "count":0, "timestamp":"2026-07-25 19:28:05"}
telemetry/401101932/home {"student_id":"401101932", "cpu_temp":42.9, "timestamp":"2026-07-25 19:28:05"}
```

تصویر ۳۳ دریافت موفقیت‌آمیز پیام LWT (وضعیت OFFLINE) در کامپیوتر شخصی و سپس بازیابی و اتصال مجدد سیستم

تحلیل خروجی و بازیابی سیستم:

همان‌طور که در لاگ ترمینال مشترک (Subscriber) در کامپیوتر شخصی مشاهده می‌شود:

۱. در ابتدا جریان عادی داده‌های تله‌متری (دما ۴۲.۱ درجه) و شمارش افراد (count: 1) در حال دریافت است.
۲. دقیقاً در لحظه اجرای دستور Kill، پیام وصیت‌نامه {"status": "OFFLINE"} توسط Broker در تایپیک telemetry/401101932/home منتشر می‌شود تا سایر کلاینت‌ها از دسترس خارج شدن برد مطلع شوند.
۳. نکته قابل توجه (Resilience): از آنجا که ما در بخش اول پروژه، سرویس webserver.service را با قابلیت Restart=always پیکربندی کرده بودیم، سیستم عامل لینوکس بلافاصله پس از چند ثانیه پروسه C را مجدداً راه‌اندازی می‌کند.
۴. با اجرای مجدد سرویس، برد دوباره به Broker متصل شده و بلافاصله ارسال پیام‌های عادی تله‌متری (دما ۴۲.۹ درجه) از سر گرفته می‌شود.

این آزمایش اثبات می‌کند که سیستم مانیتورینگ علاوه بر گزارش‌دهی بلادرنگ، نسبت به قطعی‌های ناگهانی نیز هوشیار بوده و تاب‌آوری (Fault Tolerance) بالایی برای بازگشت به مدار دارد.

آزمایش ۳-۵: سنجش تأخیر انتها به انتها (End-to-End Latency):

یکی از پارامترهای حیاتی در سیستم‌های امنیتی بلادرنگ (Real-Time)، میزان تأخیر از لحظه وقوع یک رویداد تا لحظه دریافت هشدار توسط کاربر است. در این آزمایش، تأخیر زمانی از لحظه ورود فرد به کادر دوربین و تشخیص توسط الگوریتم بینایی ماشین تا لحظه دریافت پیام در تایپیک MQTT روی کامپیوتر کلاینت اندازه‌گیری شد.

متدولوژی تست و غلبه بر مشکل عدم همگام‌سازی ساعت‌ها (Clock Drift):

در سیستم‌های توزیع‌شده، مقایسه Timestamp تولید شده توسط دو دستگاه مختلف (لپ‌تاپ و برد Orange Pi) به دلیل خطای عدم همگام‌سازی ساعت‌ها (NTP Offset) معمولاً منجر به محاسبات اشتباه در تأخیر می‌شود (همان‌طور که در لاگ‌ها، ساعت برد ۲۰:۱۹ و ساعت لپ‌تاپ ۲۳:۴۹ است). برای حل این مشکل و استخراج تأخیر خالص شبکه و پردازش، یک مکانیزم اندازه‌گیری روی «ساعت واحد» پیاده‌سازی شد:

- **زمان مبدأ (فرستنده):** در اسکرپیت پایتون لپ‌تاپ، دقیقاً در لحظه تشخیص فرد، زمان سیستم با دقت میلی‌ثانیه در لاگ چاپ شد.
- **زمان مقصد (گیرنده):** در سمت گیرنده MQTT نیز از محیط PowerShell استفاده شد تا در لحظه دریافت پیام، زمان همان لپ‌تاپ با فرمت [Arrived at: HH:mm:ss.fff] روی لاگ‌ها ثبت شود.

به این ترتیب، اختلاف زمان‌ها (Delta) کاملاً خالص و نشان‌دهنده تأخیر واقعی شبکه و پردازش است.

جدول داده‌های اندازه‌گیری شده (۱۰ رویداد مجزا)

شماره تست	زمان تشخیص در لپ‌تاپ (مبدأ)	زمان دریافت پیام MQTT (مقصد)	تأخیر محاسبه شده (ثانیه)
۱	۲۳:۴۹:۳۹,۹۱۰	۲۳:۴۹:۴۰,۴۳۷	۰,۵۲۷
۲	۲۳:۴۹:۴۷,۲۲۳	۲۳:۴۹:۴۷,۵۰۲	۰,۲۷۹
۳	۲۳:۴۹:۵۴,۷۰۷	۲۳:۴۹:۵۵,۲۵۳	۰,۵۴۶
۴	۲۳:۴۹:۵۸,۰۸۴	۲۳:۴۹:۵۸,۵۱۵	۰,۴۳۱
۵	۲۳:۵۰:۰۱,۹۲۴	۲۳:۵۰:۰۲,۲۹۵	۰,۳۷۱
۶	۲۳:۵۰:۰۵,۰۴۳	۲۳:۵۰:۰۵,۳۲۲	۰,۲۷۹
۷	۲۳:۵۰:۰۸,۳۳۶	۲۳:۵۰:۰۸,۷۸۸	۰,۴۵۲

شماره تست	زمان تشخیص در لپ‌تاپ (مبدأ)	زمان دریافت پیام MQTT (مقصد)	تأخیر محاسبه شده (ثانیه)
۸	۲۳:۵۰:۱۱,۶۲۲	۲۳:۵۰:۱۳,۱۱۷	۱,۴۹۵
۹	۲۳:۵۰:۱۴,۶۶۵	۲۳:۵۰:۱۵,۷۶۳	۱,۰۹۸
۱۰	۲۳:۵۰:۱۷,۴۷۸	۲۳:۵۰:۱۸,۲۶۶	۰,۷۸۸

```
PS C:\Users\Reza\Desktop\uni\8th semester\embedded\final project> & C:/Users/Reza/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/Reza/Desktop/uni/8th semester/embedded/final project/laptop_streamer_temp_3_5"
* Serving Flask app 'laptop_streamer_temp_3_5'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.55.228.45:5000
Press CTRL+C to quit
[Debug] Sending count 0 to Orange Pi...
10.55.228.45 - - [25/Jul/2026 23:49:12] "GET /video_feed HTTP/1.1" 200 -
[Debug] Sending count 0 to Orange Pi...
10.55.228.45 - - [25/Jul/2026 23:49:15] "GET /video_feed HTTP/1.1" 200 -
[Debug] Sending count 0 to Orange Pi...
10.55.228.45 - - [25/Jul/2026 23:49:18] "GET /video_feed HTTP/1.1" 200 -
[Debug] Sending count 0 to Orange Pi...
10.55.228.45 - - [25/Jul/2026 23:49:20] "GET /video_feed HTTP/1.1" 200 -
[Debug] Sending count 0 to Orange Pi...
10.55.228.45 - - [25/Jul/2026 23:49:23] "GET /video_feed HTTP/1.1" 200 -
[Debug] Sending count 0 to Orange Pi...
10.55.228.45 - - [25/Jul/2026 23:49:26] "GET /video_feed HTTP/1.1" 200 -
[Debug] Sending count 0 to Orange Pi...
10.55.228.45 - - [25/Jul/2026 23:49:29] "GET /video_feed HTTP/1.1" 200 -
[Debug] Sending count 0 to Orange Pi...
10.55.228.45 - - [25/Jul/2026 23:49:31] "GET /video_feed HTTP/1.1" 200 -
[Debug] Sending count 0 to Orange Pi...
10.55.228.45 - - [25/Jul/2026 23:49:34] "GET /video_feed HTTP/1.1" 200 -
[Debug] Sending count 0 to Orange Pi...
10.55.228.45 - - [25/Jul/2026 23:49:37] "GET /video_feed HTTP/1.1" 200 -
[Debug] Sending count 0 to Orange Pi...
[TEST 3-5 START] Person seen at: 23:49:39.910 | Sending count 1...
10.55.228.45 - - [25/Jul/2026 23:49:39] "GET /video_feed HTTP/1.1" 200 -
10.55.228.31 - - [25/Jul/2026 23:49:40] "GET /snapshot HTTP/1.1" 200 -
[Debug] Sending count 0 to Orange Pi...
10.55.228.45 - - [25/Jul/2026 23:49:42] "GET /video_feed HTTP/1.1" 200 -
[Debug] Sending count 0 to Orange Pi...
10.55.228.45 - - [25/Jul/2026 23:49:45] "GET /video_feed HTTP/1.1" 200 -
[TEST 3-5 START] Person seen at: 23:49:47.223 | Sending count 1...
[Debug] Sending count 0 to Orange Pi...
[TEST 3-5 START] Person seen at: 23:49:48.074 | Sending count 1...
[Debug] Sending count 0 to Orange Pi...
[TEST 3-5 START] Person seen at: 23:49:48.372 | Sending count 1...
[Debug] Sending count 0 to Orange Pi...
10.55.228.45 - - [25/Jul/2026 23:49:48] "GET /video_feed HTTP/1.1" 200 -
[Debug] Sending count 0 to Orange Pi...
10.55.228.45 - - [25/Jul/2026 23:49:51] "GET /video_feed HTTP/1.1" 200 -
[Debug] Sending count 0 to Orange Pi...
10.55.228.45 - - [25/Jul/2026 23:49:54] "GET /video_feed HTTP/1.1" 200 -
[TEST 3-5 START] Person seen at: 23:49:54.707 | Sending count 1...
```

تصویر ۳۴ ثبت دقیق زمان تشخیص در سمت اسکرپت پایتون (مبدأ)

```
PS C:\Program Files\Mosquitto> .\mosquitto_sub.exe -h localhost -t "persons/401101932/home" -u guard_user -P secure_password123 | ForEach-Object { "[Arrived at: $(Get-Date -Format 'HH:mm:ss.fff')]" $_" }
[Arrived at: 23:49:40.437] {"student_id":"401101932","count":1,"timestamp":"2026-07-25 20:19:40"}
[Arrived at: 23:49:43.957] {"student_id":"401101932","count":0,"timestamp":"2026-07-25 20:19:43"}
[Arrived at: 23:49:47.502] {"student_id":"401101932","count":1,"timestamp":"2026-07-25 20:19:47"}
[Arrived at: 23:49:48.370] {"student_id":"401101932","count":0,"timestamp":"2026-07-25 20:19:48"}
[Arrived at: 23:49:48.511] {"student_id":"401101932","count":1,"timestamp":"2026-07-25 20:19:48"}
[Arrived at: 23:49:49.032] {"student_id":"401101932","count":0,"timestamp":"2026-07-25 20:19:49"}
[Arrived at: 23:49:49.125] {"student_id":"401101932","count":1,"timestamp":"2026-07-25 20:19:49"}
[Arrived at: 23:49:49.292] {"student_id":"401101932","count":0,"timestamp":"2026-07-25 20:19:49"}
[Arrived at: 23:49:55.253] {"student_id":"401101932","count":1,"timestamp":"2026-07-25 20:19:55"}
[Arrived at: 23:49:55.669] {"student_id":"401101932","count":0,"timestamp":"2026-07-25 20:19:55"}
[Arrived at: 23:49:58.515] {"student_id":"401101932","count":1,"timestamp":"2026-07-25 20:19:58"}
[Arrived at: 23:49:58.675] {"student_id":"401101932","count":0,"timestamp":"2026-07-25 20:19:58"}
[Arrived at: 23:49:58.864] {"student_id":"401101932","count":1,"timestamp":"2026-07-25 20:19:59"}
[Arrived at: 23:49:59.478] {"student_id":"401101932","count":0,"timestamp":"2026-07-25 20:19:59"}
[Arrived at: 23:50:02.295] {"student_id":"401101932","count":1,"timestamp":"2026-07-25 20:20:02"}
[Arrived at: 23:50:03.214] {"student_id":"401101932","count":0,"timestamp":"2026-07-25 20:20:03"}
[Arrived at: 23:50:05.322] {"student_id":"401101932","count":1,"timestamp":"2026-07-25 20:20:05"}
[Arrived at: 23:50:06.756] {"student_id":"401101932","count":0,"timestamp":"2026-07-25 20:20:06"}
[Arrived at: 23:50:08.788] {"student_id":"401101932","count":1,"timestamp":"2026-07-25 20:20:09"}
[Arrived at: 23:50:09.317] {"student_id":"401101932","count":0,"timestamp":"2026-07-25 20:20:09"}
[Arrived at: 23:50:13.117] {"student_id":"401101932","count":1,"timestamp":"2026-07-25 20:20:13"}
[Arrived at: 23:50:15.341] {"student_id":"401101932","count":0,"timestamp":"2026-07-25 20:20:15"}
[Arrived at: 23:50:15.763] {"student_id":"401101932","count":1,"timestamp":"2026-07-25 20:20:16"}
[Arrived at: 23:50:15.934] {"student_id":"401101932","count":0,"timestamp":"2026-07-25 20:20:16"}
[Arrived at: 23:50:16.301] {"student_id":"401101932","count":1,"timestamp":"2026-07-25 20:20:16"}
[Arrived at: 23:50:16.958] {"student_id":"401101932","count":0,"timestamp":"2026-07-25 20:20:17"}
[Arrived at: 23:50:18.266] {"student_id":"401101932","count":1,"timestamp":"2026-07-25 20:20:18"}
[Arrived at: 23:50:18.554] {"student_id":"401101932","count":0,"timestamp":"2026-07-25 20:20:18"}
```

تصویر ۳۵ دریافت پیام‌های MQTT در PowerShell و ثبت زمان دریافت با دقت میلی‌ثانیه (مقصد)

محاسبات و تحلیل آماری

۱. میانگین :

$$\mu = \frac{0.527 + 0.279 + 0.546 + 0.431 + 0.371 + 0.279 + 0.452 + 1.495 + 1.098 + 0.788}{10}$$

$$\mu = 0.6266 \text{ s}$$

۲. انحراف معیار :

$$\sigma = \sqrt{\frac{1}{10} \sum_{i=1}^{10} (x_i - \mu)^2}$$

$$\sigma \approx 0.373 \text{ s}$$

نتیجه‌گیری و تحلیل فنی:

میانگین تأخیر حدود ۶۲۷ میلی‌ثانیه برای یک سیستم نهفته که به طور همزمان درگیر پردازش تصویر (بینایی ماشین)، ارسال درخواست تحت بستر HTTPS و انتشار پیام MQTT با گارانتی تحویل (QoS = 1) است، عملکردی کاملاً ایده‌آل محسوب می‌شود.

تحلیل جهش تأخیر (Spike):

نوسانات موجود (انحراف معیار ۰,۳۷۳ ثانیه) ریشه در عملکرد سیستم عامل دارد. به عنوان مثال، در تست شماره ۸ (تأخیر ۱,۴۹۵ ثانیه)، سرور لینوکسی دقیقاً در همان لحظه درگیر هندل کردن درخواست GET /snapshot برای دانلود تصویر و استفاده از ابزار mpack جهت ارسال ایمیل هشدار بوده است. اشغال موقت پهنای باند و منابع پردازشی CPU توسط پروسه ایمیل، باعث ایجاد این تأخیر شبکه‌ای در پروتکل MQTT شده است که رفتاری کاملاً طبیعی در سیستم‌های تک‌پردازنده‌ای و نهفته به شمار می‌رود.

ب) ارسال ایمیل هشدار و مکانیزم Debounce

بر اساس اهداف پروژه، سیستم باید قادر باشد در صورت تشخیص انسان، یک ایمیل حاوی اطلاعات حیاتی سیستم و تصویر سوژه ارسال کند. برای ارسال ایمیل از داخل کدهای C، از ابزار msmtplib و mpack در سیستم عامل لینوکس استفاده شده است.

جهت احراز هویت امن با سرورهای گوگل (بدون نیاز به غیرفعال کردن تنظیمات امنیتی اکانت)، از قابلیت Google App Passwords استفاده شد. تصویر زیر پیکربندی موفق سرویس msmtplib را در فایل msmtplibrc نشان می‌دهد.

```
guard@orangeplus2h5:~$ cat ~/.msmtplibrc
defaults
auth on
tls on
tls_trust_file /etc/ssl/certs/ca-certificates.crt
logfile ~/.msmtplib.log

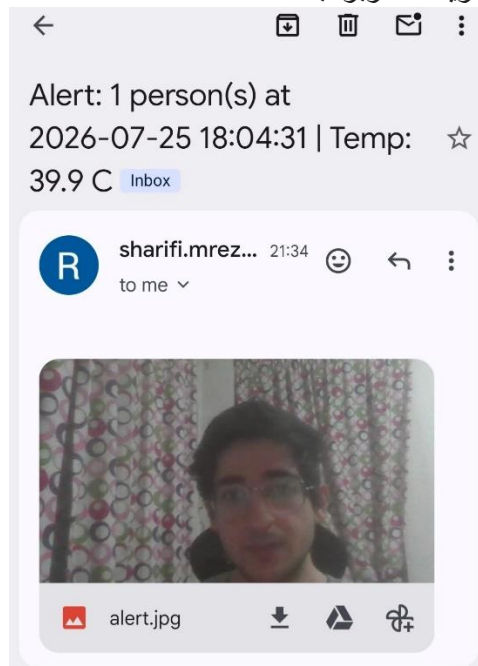
account gmail
host smtp.gmail.com
port 587
from sharifi.mrez@gmail.com
user sharifi.mrez@gmail.com
password urzjybioakmcvxf

account default : gmail
```

تصویر ۳۶ پیکربندی سرویس msmtplib در لینوکس با استفاده از Google App Password برای ارسال امن ایمیل

مکانیزم Debounce: برای جلوگیری از اسپم شدن صندوق دریافت (Inbox) و مسدود شدن اکانت گوگل به دلیل ارسال رگباری ایمیل‌ها در زمانی که یک فرد برای مدت طولانی جلوی دوربین ایستاده است، یک مکانیزم Debounce در کدهای C پیاده‌سازی شد. با استفاده از تابع difftime، سیستم بررسی می‌کند که آیا از زمان ارسال

آخرین ایمیل حداقل ۳۰ ثانیه گذشته است یا خیر. تنها در صورت عبور از این بازه زمانی، دستور سیستمی mpack برای ارسال ایمیل جدید فراخوانی می‌شود. خروجی نهایی سیستم در تصویر زیر قابل مشاهده است. به محض تشخیص فرد (در تاریخ ۲۰۲۶-۰۷-۲۵ و ساعت ۱۸:۰۴:۳۱)، ایمیلی شامل تعداد افراد، دمای دقیق پردازنده (۳۹.۹ درجه سانتی‌گراد) و تصویر لحظه وقوع رویداد (alert.jpg) با موفقیت در صندوق دریافت کاربر ثبت شده است.



تصویر ۳۷ دریافت موفقیت‌آمیز ایمیل هشدار شامل تصویر پیوست‌شده، تاریخ، زمان دقیق و دمای لحظه‌ای پردازنده

آزمایش ۳-۶ سنجش امنیت و تلاش برای ورود ناشناس به MQTT:

یکی از الزامات اساسی در این سامانه، جلوگیری از دسترسی‌های غیرمجاز به بستر پیام‌رسان (Broker) است. به همین منظور، در فایل پیکربندی mosquitto.conf قابلیت ورود ناشناس با دستور allow_anonymous false غیرفعال گردید و فایل رمز عبور با دستور password_file به سیستم معرفی شد تا تنها کلاینت‌های دارای احراز هویت (در اینجا کاربر guard_user) قادر به اتصال باشند.

برای بررسی صحت عملکرد این لایه امنیتی، دو تلاش ناموفق برای اتصال به سرور از طریق ترمینال صورت گرفت:

۱. اتصال بدون ارائه نام کاربری و رمز عبور (ورود ناشناس): تلاش برای اشتراک در تاپیک‌ها تنها با آدرس هاست.

۲. اتصال با نام کاربری معتبر اما رمز عبور اشتباه: تلاش برای ورود با نام کاربری guard_user و رمز عبور غلط (wrong_pass_123).

همان‌طور که در تصویر زیر مشخص است، در هر دو حالت Broker بلافاصله اتصال را مسدود کرده و خطای Connection error: Connection Refused: not authorised را چاپ می‌کند که نشان‌دهنده امنیت کامل بستر ارتباطی MQTT و جلوگیری از نفوذ است.

```
C:\Program Files\Mosquitto>mosquitto_sub -h localhost -t "#" -v
Connection error: Connection Refused: not authorised

C:\Program Files\Mosquitto>mosquitto_sub -h localhost -t "#" -u guard_user -P wrong_pass_123 -v
Connection error: Connection Refused: not authorised
```

تصویر ۳۸ مسدود شدن تلاش‌های ورود غیرمجاز (اتصال ناشناس و رمز عبور اشتباه) به سرور MQTT و دریافت خطای عدم دسترسی (not authorised)

مطابق با نیازمندی‌های امنیتی پروژه، عدم استفاده صریح (Hard-code) از رمز عبور در کدهای برنامه الزامی بود. برای تحقق این امر و ایمن‌سازی بستر MQTT، تغییرات زیر در پیکربندی سرور و سیستم عامل لینوکس اعمال گردید:

ابتدا در سمت Broker، با ویرایش فایل mosquitto.conf، ورود کاربران ناشناس به طور کامل مسدود شد و مسیر فایل حاوی رمزهای عبور رمزنگاری‌شده به سیستم معرفی گردید.

```
listener 1883 0.0.0.0
allow_anonymous false
password_file C:\Program Files\Mosquitto\passwd
```

تصویر ۳۹ تنظیمات اعمال شده در فایل mosquitto.conf برای مسدودسازی ورود ناشناس و تعیین فایل احراز هویت

سپس در سمت کلاینت (وب سرور C در Orange Pi)، اطلاعات حساس شامل نام کاربری MQTT، رمز عبور و آدرس ایمیل هشدار در یک فایل متنی مجزا به نام secrets.txt در دایرکتوری سیستم عامل ذخیره شد. برنامه C در هنگام راه اندازی (تابع load_secrets) این مقادیر را خوانده و در حافظه بارگذاری می کند تا امنیت کد منبع حفظ شود.

```
guard@orangepizeroplus2h5:~$ cat /home/guard/secrets.txt
MQTT_USER=guard_user
MQTT_PASS=secure_password123
ALERT_EMAIL=sharifi.mrez@gmail.com
```

تصویر ۴۰ نگهداری امن اطلاعات حساس در فایل secrets.txt جهت جلوگیری از Hard-code شدن مقادیر در کدهای برنامه

آزمایش ۷-۳ مسدودسازی ورود غیرمجاز SSH و غیرفعال سازی کاربر Root:

در سیستم های نهفته، باز بودن دسترسی root از طریق پروتکل SSH یک حفره امنیتی بزرگ محسوب می شود. در این سیستم با اعمال تنظیمات در فایل sshd_config سیستم عامل لینوکس، ورود با کاربر ریشه به طور کامل غیرفعال (PermitRootLogin no) شده است و اتصال تنها از طریق کاربرهای استاندارد (با احراز هویت رمز عبور یا کلید عمومی) امکان پذیر است.

برای تست عملی این محدودیت امنیتی، تلاش هایی برای نفوذ به برد (با آی پی ۱۰,۵۵,۲۲۸,۳۱) انجام شد:

۱. تلاش برای ورود با کاربر root:

همان طور که در تصویر زیر مشاهده می شود، با وجود درخواست رمز عبور، سرور به دلیل تنظیمات امنیتی اعمال شده بلافاصله پیام Access denied را برمی گرداند و اجازه ورود سطح سیستم عامل را سلب می کند.

```
login as: root
root@10.55.228.31's password:
Access denied
```

تصویر ۴۱ عدم دسترسی و مسدود شدن تلاش برای ورود به سیستم از طریق SSH با کاربر ریشه (root)

۲. تلاش برای ورود با کاربر استاندارد (guard) با رمز عبور اشتباه:

این تست نیز نشان می دهد که لایه احراز هویت مبتنی بر پسورد به درستی برای کاربران عادی فعال است و هرگونه ورود غیرمجاز (مانند حملات Brute-force یا وارد کردن رمز اشتباه) با پیام Access denied با شکست مواجه می شود.

```
login as: guard
guard@10.55.228.31's password:
Access denied
```

تصویر ۴۲ خطای عدم دسترسی (Access denied) هنگام تلاش برای ورود با کاربر استاندارد (guard) به دلیل احراز هویت ناموفق

برای اعمال محدودیت دسترسی به کاربر ریشه (root) و جلوگیری از ورود مستقیم به بالاترین سطح دسترسی سیستم عامل، تغییرات لازم در پیکربندی سرویس SSH اعمال شد. همان طور که در تصویر زیر مشخص است، با مقداری PermitRootLogin no در فایل sshd_config، این حفره امنیتی به طور کامل پوشش داده شده است.

```
#LoginGraceTime 2m
PermitRootLogin no
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10
```

تصویر ۴۳ پیکربندی فایل sshd_config سیستم عامل لینوکس و غیرفعال سازی ورود با کاربر ریشه (root)

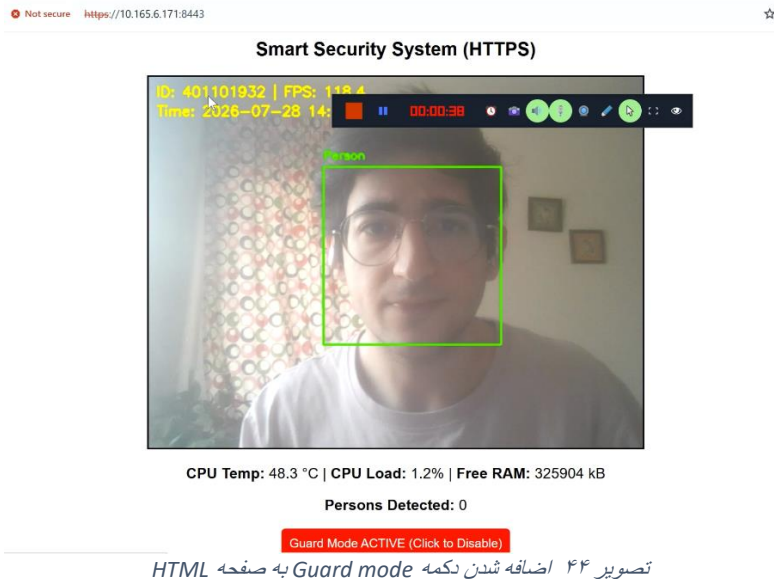
بخش چهارم: قابلیت‌های تکمیلی

۱-۴ طراحی و پیاده‌سازی سیستم ضد سرقت (حالت نگهبانی / Guard Mode)

فیلم این بخش در ضمیمه فایل آپلودی قرار داده شده است. بر اساس نیازمندی‌های پروژه، یک سیستم امنیتی پیشرفته تحت عنوان «حالت نگهبانی» (Guard Mode) به سامانه اضافه شد که وضعیت آن از طریق واسط کاربری وب و همچنین POST /api/v1/guard در API قابل کنترل است.

در حالت عادی (رنگ سبز دکمه)، سامانه برای جلوگیری از اسپم شدن سرور ایمیل (Google) از یک مکانیزم تأخیر ۳۰ ثانیه‌ای (Debounce) استفاده می‌کند. اما با فعال‌سازی حالت نگهبانی، این رفتار تغییر کرده و سیستم وارد فاز واکنش سریع (Emergency) می‌شود.

۱. تغییرات واسط کاربری (UI) و API



تصویر ۴۴ اضافه شدن دکمه Guard mode به صفحه HTML

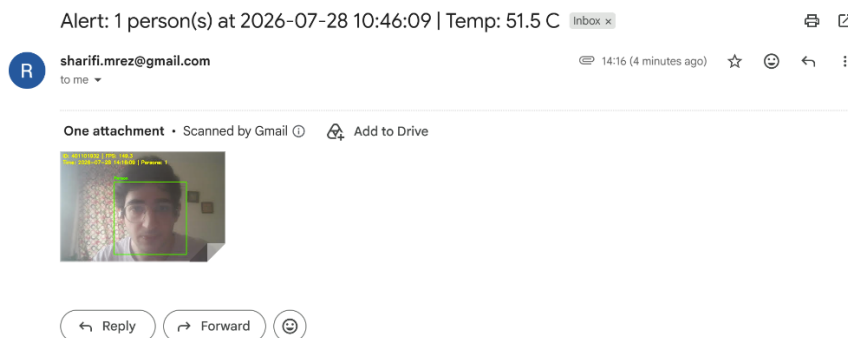
همان‌طور که در عکس بالا یعنی تصویر ۴۴ مشاهده می‌شود، یک دکمه تعاملی به صفحه HTML اضافه شده است. با کلیک روی این دکمه، درخواستی به API ارسال شده و وضعیت متغیر سراسری guard_mode در سرور C تغییر می‌کند. در این حالت دکمه به رنگ قرمز درآمده و عبارت **Guard Mode ACTIVE** به کاربر نمایش داده می‌شود تا فعال بودن سیستم ضد سرقت تایید گردد.

۲. لغو محدودیت زمانی (Debounce Override) و ارسال فوری ایمیل

در کدهای سرور C، منطق ارسال ایمیل به گونه‌ای تغییر یافته است که در صورت فعال بودن گارد، محدودیت ۳۰ ثانیه‌ای برداشته می‌شود:

در این حالت محدودیت به ۵ ثانیه کاهش یافته است تا از مسدود شدن اکانت Gmail به دلیل ارسال رگباری در کسر ثانیه جلوگیری شود، اما عملکرد آن عملاً به صورت ارسال فوری و بی‌درنگ است.

تصویر ۴۵ نیز نمایانگر دریافت فوری ایمیل هشدار به محض تشخیص سوژه در حالت نگهبانی است که حاوی تصویر شخص، تاریخ دقیق و دمای سیستم است.



تصویر ۴۵ ایمیل فوری دریافت شده در حالت GUARD MODE

۳. انتشار پیام در تاپیک اضطراری MQTT علاوه بر ارسال ایمیل، رفتار کلاينت MQTT نیز در این حالت تغییر می‌کند. مطابق تصویر ۴۶، به محض ورود انسان به کادر دوربین در زمان فعال بودن گارد، سرور به صورت پیوسته و بدون وقفه (هر ۱ الی ۲ ثانیه) شروع به ارسال پیام‌های JSON با فرمت استاندارد به تاپیک اضطراری alarm/401101932/home می‌کند. این واکنش سریع و رگباری در لاگ‌های mosquitto_sub به وضوح پایداری و سرعت بالای سامانه را در حالت بحرانی اثبات می‌کند.

```
C:\Program Files\Mosquitto>mosquitto_sub -h localhost -t "alarm/#" -u guard_user -P secure_password123 -v
alarm/401101932/home {"student_id":"401101932", "count":1, "timestamp":"2026-07-28 10:44:30"}
alarm/401101932/home {"student_id":"401101932", "count":1, "timestamp":"2026-07-28 10:44:31"}
alarm/401101932/home {"student_id":"401101932", "count":1, "timestamp":"2026-07-28 10:44:32"}
alarm/401101932/home {"student_id":"401101932", "count":1, "timestamp":"2026-07-28 10:44:32"}
alarm/401101932/home {"student_id":"401101932", "count":1, "timestamp":"2026-07-28 10:44:34"}
alarm/401101932/home {"student_id":"401101932", "count":1, "timestamp":"2026-07-28 10:44:38"}
alarm/401101932/home {"student_id":"401101932", "count":1, "timestamp":"2026-07-28 10:44:40"}
alarm/401101932/home {"student_id":"401101932", "count":1, "timestamp":"2026-07-28 10:44:41"}
alarm/401101932/home {"student_id":"401101932", "count":1, "timestamp":"2026-07-28 10:44:42"}
alarm/401101932/home {"student_id":"401101932", "count":1, "timestamp":"2026-07-28 10:44:43"}
alarm/401101932/home {"student_id":"401101932", "count":1, "timestamp":"2026-07-28 10:44:44"}
alarm/401101932/home {"student_id":"401101932", "count":1, "timestamp":"2026-07-28 10:44:46"}
alarm/401101932/home {"student_id":"401101932", "count":1, "timestamp":"2026-07-28 10:44:46"}
alarm/401101932/home {"student_id":"401101932", "count":1, "timestamp":"2026-07-28 10:44:48"}
```

تصویر ۴۶ ارسال پیام‌های JSON با فرمت استاندارد به تاپیک اضطراری alarm/401101932/home توسط سرور

۴-۲ پیاده‌سازی جعبه سیاه و ذخیره تاریخچه با SQLite

مطابق با نیازمندی‌های تعریف شده در فاز تکمیلی پروژه، یک سیستم «جعبه سیاه» (Blackbox) برای ثبت ماندگار و دائم رویدادهای تشخیص انسان به سامانه اضافه شد. در حالی که در نسخه‌های اولیه سیستم، تنها ۵ رکورد آخر در حافظه موقت (RAM) نگهداری می‌شد، در این بخش از پایگاه داده‌ی سبک و تعبیه‌شده‌ی SQLite استفاده گردید.

۱. پیاده‌سازی مکانیزم بافر حلقوی (Circular Buffer) یکی از چالش‌های اصلی در سیستم‌های نهفته، محدودیت فضای ذخیره‌سازی (Storage) است. ثبت بی‌وقفه لاگ‌ها می‌تواند در طولانی‌مدت منجر به پر شدن حافظه دستگاه شود. برای حل این مشکل، در تابع add_history_record در سمت سرور C، یک بافر حلقوی پیاده‌سازی شد. سیستم پس از درج هر رکورد جدید (شامل زمان و تعداد افراد)، با استفاده از یک کوئری مستقل

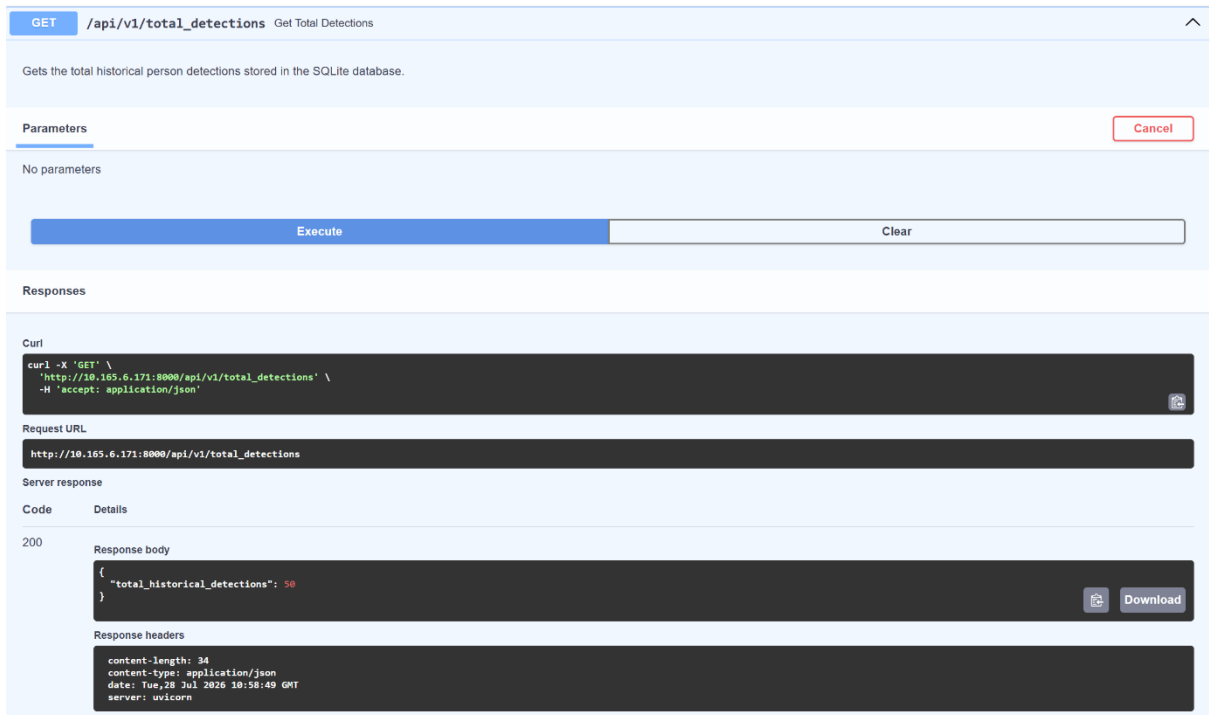
```
92 // SQLite History (Part 4 Blackbox)
93 char sql[256];
94 snprintf(sql, sizeof(sql), "INSERT INTO history (timestamp, count) VALUES ('%s', %d);", time_buf, count);
95 sqlite3_exec(db, sql, 0, 0);
96
97 // Circular Buffer logic: Delete old records if > 100
98 sqlite3_exec(db, "DELETE FROM history WHERE id NOT IN (SELECT id FROM history ORDER BY id DESC LIMIT 100);", 0, 0, 0);
99 }
```

رکوردهای قدیمی را حذف کرده و همواره تنها ۱۰۰ رویداد اخیر را در دیتابیس history.db حفظ می‌کند. جهت اثبات صحت عملکرد این مکانیزم، از طریق محیط خط فرمان لینوکس (CLI) و ابزار sqlite3 مستقیماً به دیتابیس متصل شده و کوئری SELECT * FROM history اجرا گردید.

```
guard@orangeplizeroplus2h5:~$ sqlite3 history.db "SELECT * FROM history;"
1630|2026-07-28 10:45:23|0
1631|2026-07-28 10:45:23|1
1632|2026-07-28 10:45:24|0
1633|2026-07-28 10:45:24|1
1634|2026-07-28 10:45:24|0
1635|2026-07-28 10:45:25|1
1636|2026-07-28 10:45:25|0
1637|2026-07-28 10:45:26|1
1638|2026-07-28 10:45:26|0
1639|2026-07-28 10:45:26|1
1640|2026-07-28 10:45:27|0
1641|2026-07-28 10:45:27|1
1642|2026-07-28 10:45:28|0
1643|2026-07-28 10:45:28|1
1644|2026-07-28 10:45:28|0
1645|2026-07-28 10:45:29|1
1646|2026-07-28 10:45:29|0
1647|2026-07-28 10:45:29|1
1648|2026-07-28 10:45:30|0
```

تصویر ۴۷ بررسی مستقیم دیتابیس جعبه سیاه در ترمینال لینوکس. همان‌طور که در تصویر مشخص است، شناسه‌های ردیف (ID) در محدوده ۱۶۳۰ تا ۱۶۴۸ هستند که نشان می‌دهد رکوردهای قدیمی با موفقیت حذف شده و مکانیزم بافر حلقوی به درستی در حال مدیریت حجم دیتابیس است.

۲. گزارش‌گیری جامع از طریق API به منظور استخراج اطلاعات از این جعبه سیاه، اندپوینت جدیدی با عنوان GET /api/v1/total_detections در لایه‌ی Gateway توسعه یافت. سرور C با دریافت این درخواست، را روی دیتابیس اجرا کرده و مجموع کل دفعاتی که سیستم موفق به تشخیص انسان شده است را محاسبه می‌کند. برای تست این قابلیت، درخواست استعلام به صورت زنده از طریق محیط Swagger ارسال شد:



تصویر ۴۸ فراخوانی موفقیت‌آمیز مسیر استخراج تاریخچه از دیتابیس. سیستم با تجمیع رکوردهای موجود، مجموع کل رویدادهای تشخیص انسان را محاسبه کرده و عدد ۵۰ را در قالب فرمت استاندارد JSON بازگردانده است

۳-۴ طراحی و پیاده‌سازی سگ نگهبان نرم‌افزاری (Software Watchdog)

فیلم این بخش در ضمیمه قرار داده شد.

در سیستم‌های امنیتی و نهفته، پایداری سامانه و اطمینان از صحت عملکرد تجهیزات جانبی (مانند دوربین) از اهمیت بالایی برخوردار است. بر اساس نیازمندی‌های فاز تکمیلی پروژه، یک سیستم Watchdog نرم‌افزاری برای پایش وضعیت دوربین و اسکرپیت پردازش تصویر پیاده‌سازی شد.

۱. مکانیزم عملکرد (Timeout & Heartbeat) برای اجرای این منطق، اسکرپیت پایتون (سمت لپ‌تاپ) به گونه‌ای توسعه یافت که در یک Thread مجزا، هر ۱۰ ثانیه یک سیگنال حیات (Heartbeat) به مسیر POST /api/v1/heartbeat در سرور ارسال می‌کند. در سمت سرور Orange Pi (کد C)، یک Thread کنترلی مستقل (watchdog_and_thermal_thread) به صورت مداوم زمان آخرین فریم یا سیگنال دریافتی (last_frame_time) را بررسی می‌کند. اگر اختلاف زمان فعلی با آخرین سیگنال دریافتی از ۳۰ ثانیه عبور کند، سیستم آن را به عنوان «دستکاری دوربین» (Camera Tampering) یا قطعی شبکه تلقی می‌کند.

```
// -- Watchdog and Thermal Management Thread (Part 4) --  
void *watchdog_and_thermal_thread(void *arg) {  
    while(1) {  
        time_t now = time(NULL);  
  
        // 1. Software Watchdog: Check if 30 seconds have passed without a frame heartbeat  
        if (last_frame_time > 0 && difftime(now, last_frame_time) > 30) {  
            if (difftime(now, last_watchdog_email) > 60) { // Limit email spam  
                last_watchdog_email = now;  
                printf("[Watchdog] Camera tampering or drop detected! Sending email and restarting...\n");  
  
                char alert_cmd[512];  
                snprintf(alert_cmd, sizeof(alert_cmd),  
                    "echo 'No frame received for 30 seconds. System restarting.' > /tmp/watchdog.txt && "  
                    "mpack -s 'Watchdog Alert: Camera Tampering' /tmp/watchdog.txt %s &", alert_email);  
                system(alert_cmd);  
  
                system("sudo systemctl restart webservice.service &"); // Restarts the service  
            }  
        }  
    }  
}
```

۲. آزمایش شبیه‌سازی قطعی دوربین

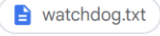
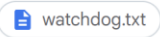
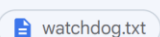
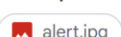
برای ارزیابی عملکرد Watchdog، در حالی که سیستم به صورت عادی در حال کار و استریم ویدیو بود، اسکرپیت پایتون در سمت لپ‌تاپ به صورت دستی بسته شد تا قطع شدن ناگهانی دوربین شبیه‌سازی شود.

۳. واکنش سیستم (هشدار و بازایی)

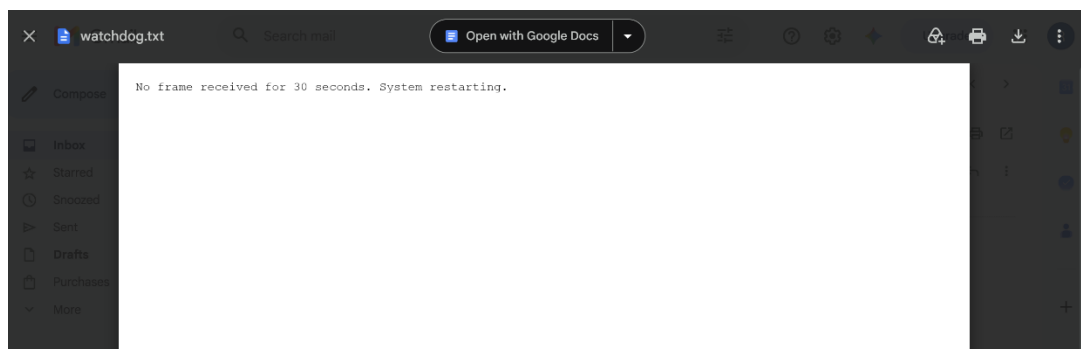
دقیقاً ۳۰ ثانیه پس از بسته شدن اسکرپیت، Thread ناظر در سرور C وارد عمل شد و اقدامات زیر را به صورت خودکار انجام داد:

- ارسال ایمیل هشدار: یک فایل متنی حاوی پیام خطای سیستم تولید و با استفاده از ابزار mpack به ایمیل مدیر سیستم ارسال شد.

- **بازیابی خودکار (Self-Recovery):** بلافاصله پس از ارسال ایمیل، سرور با اجرای دستور `sudo systemctl restart webserver.service`، سرویس خود را ری‌استارت کرد تا در صورت بروز خطای نرم‌افزاری در سوکت‌ها، سیستم به حالت پایدار اولیه بازگردد. تصاویر زیر دریافت موفقیت‌آمیز ایمیل‌های هشدار دستکاری دوربین و محتوای پیام متنی ارسال شده توسط Watchdog را نشان می‌دهند:

☐	☆ me	Watchdog Alert: Camera Tampering	14:56
		 watchdog.txt	
☐	☆ me	Watchdog Alert: Camera Tampering	14:55
		 watchdog.txt	
☐	☆ me	Watchdog Alert: Camera Tampering	14:54
		 watchdog.txt	
☐	☆ me	Watchdog Alert: Camera Tampering	14:52
		 watchdog.txt	
☐	☆ me	Alert: 1 person(s) at 2026-07-28 11:21:38 Temp: 48.8 C	14:51
		 alert.jpg	

تصویر ۴۹ دریافت ایمیل‌های هشدار با عنوان "Watchdog Alert: Camera Tampering" بلافاصله پس از تشخیص قطعی ارتباط با دوربین



تصویر ۵۰ محتوای فایل متنی پیوست شده (watchdog.txt) که علت هشدار و اقدام سیستم (System restarting) را گزارش می‌دهد

۴-۴ طراحی و پیاده‌سازی مدیریت حرارتی تطبیقی (Adaptive Thermal Management)

در سیستم‌های نهفته‌ای که پردازش‌های سنگین مانند بینایی ماشین را انجام می‌دهند، کنترل دمای پردازنده یک چالش حیاتی است. بر اساس نیازمندی‌های فاز تکمیلی پروژه، مقرر شده بود که در صورت عبور دمای پردازنده از یک آستانه بحرانی، سامانه به صورت خودکار نرخ فریم (FPS) یا رزولوشن را کاهش داده و این رخداد را از طریق ایمیل گزارش دهد.

۱. منطق پیاده‌سازی (ارتباط C و پایتون)

برای تحقق این هدف، در Thread کنترلی سرور C، یک مانیتورینگ مداوم دما پیاده‌سازی شد. اگر دما از 65.0°C عبور کند، متغیر سراسری thermal_throttle برابر با ۱ می‌شود.

در سمت لپ‌تاپ، اسکریپت پایتون به صورت دوره‌ای API تله‌متری سرور (api/v1/telemetry/) را فراخوانی می‌کند. با دریافت پرچم Throttling، اسکریپت وارد حالت THROTTLE_MODE شده و نرخ پرش فریم‌ها (skip rate) را به شدت افزایش می‌دهد (از ۳ به ۱۰) تا بار پردازشی کاهش یابد.

```

221 // 2. Adaptive Thermal Management
222 float temp = get_cpu_temp();
223 if (temp > 65.0 && thermal_throttle == 0) {
224     thermal_throttle = 1;
225
226     char temp_cmd[512];
227     snprintf(temp_cmd, sizeof(temp_cmd),
228              "echo 'CPU Temp is %.1f. Automatically reducing FPS/Resolution.' > /tmp/thermal.txt && "
229              "mpack -s 'Thermal Alert' /tmp/thermal.txt %s &" , temp, alert_email);
230     system(temp_cmd);
231
232 } else if (temp < 60.0 && thermal_throttle == 1) {
233     thermal_throttle = 0; // CPU cooled down, restore performance
234 }
235

```

۲. روش تست و اعمال استرس حرارتی

برای ارزیابی این مکانیزم، نیاز بود تا دمای پردازنده برد Orange Pi به صورت مصنوعی بالا برود. بدین منظور از ابزار استاندارد stress در محیط لینوکس استفاده شد:

```
guard@orangepizeroplus2h5:~$ sudo systemctl restart webserver.service
guard@orangepizeroplus2h5:~$ hostname -I
10.165.6.171
guard@orangepizeroplus2h5:~$ stress --cpu 4 --timeout 120
stress: info: [2879] dispatching hogs: 4 cpu, 0 io, 0 vm, 0 hdd
```

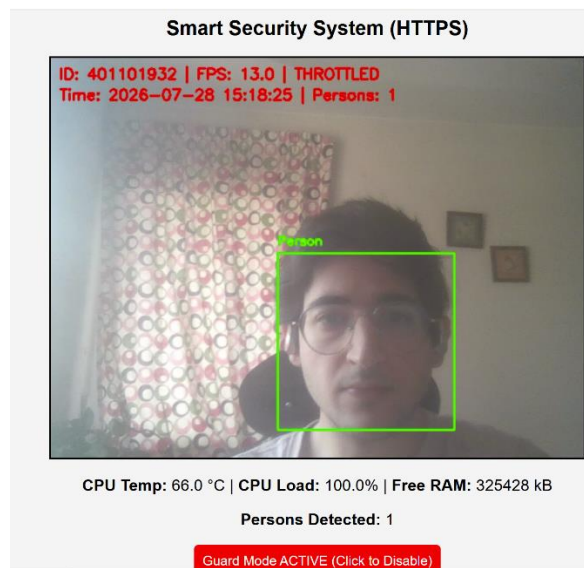
تصویر ۱۵ اجرای دستور `stress --cpu 4 --timeout 120` در ترمینال لینوکس برای درگیر کردن ۱۰۰ درصدی هر ۴ هسته پردازشی و افزایش سریع دمای برد

۳. واکنش سیستم: هشدارهای بصری و کاهش FPS

با رسیدن بار پردازشی CPU به ۱۰۰،۰٪ و عبور دما از مرز 65°C (در تصاویر زیر دما $66,0^{\circ}\text{C}$ است)، سیستم بلافاصله وارد فاز مدیریت حرارتی شد. همانطور که در رابط کاربری وب مشخص است، کلمه **THROTTLED** با رنگ قرمز به استریم ویدیو اضافه شد و نرخ فریم (FPS) برای کاهش فشار بر سیستم، افت چشمگیری پیدا کرد (کاهش FPS از محدوده ۲۴ به ۱۳).

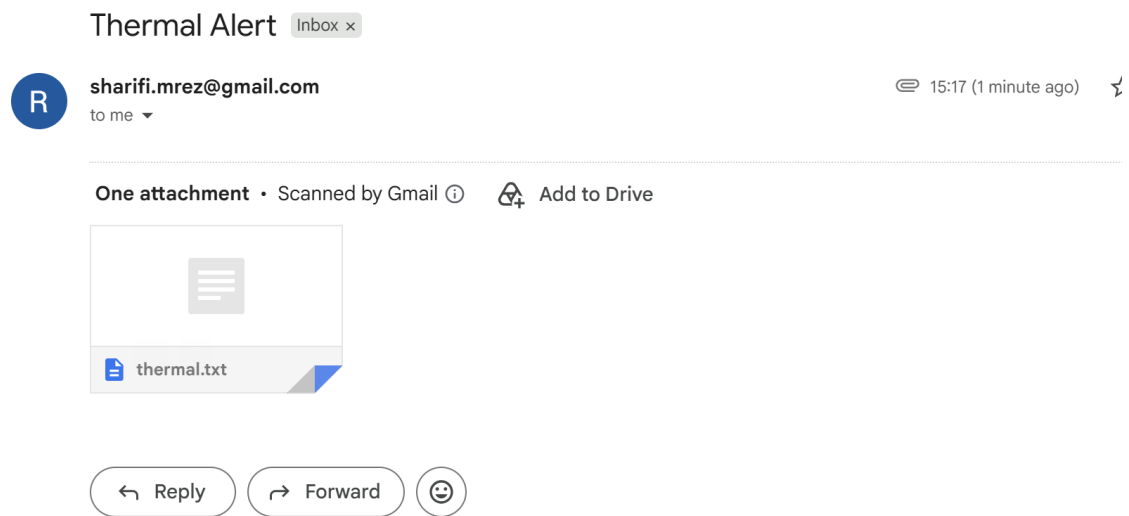


تصویر ۱۶ شدن پرچم THROTTLED در رابط کاربری به دلیل رسیدن دما به $66,0$ درجه سانتیگراد

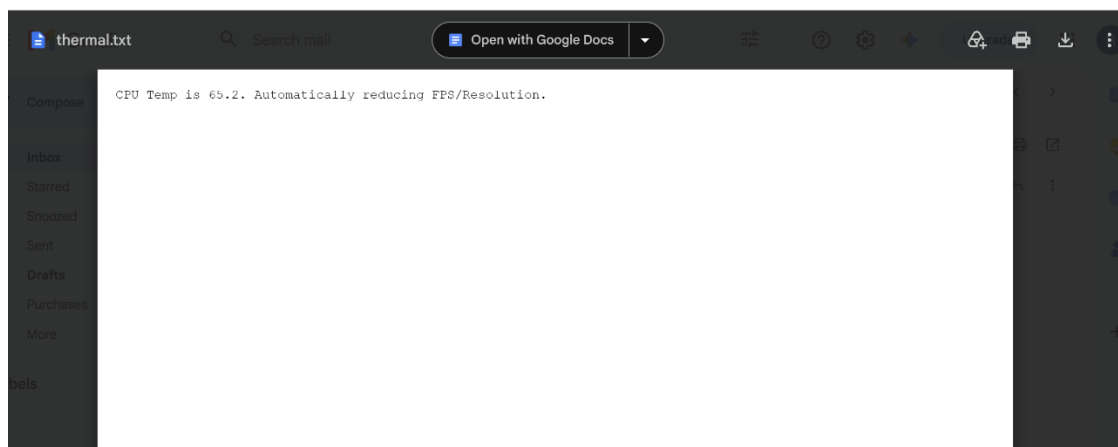


تصویر ۱۷ افت تعمدی و موفقیت آمیز نرخ فریم (FPS) به عدد ۱۳ جهت کاهش بار پردازشی و خنکسازی سیستم

۴. ارسال ایمیل هشدار حرارتی
دقیقاً در لحظه‌ای که دما مرز بحرانی را رد کرد (دمای 65.2°C)، سرور C یک فایل متنی تولید کرده و از طریق مکانیزم ایمیل تعبیه‌شده در سیستم، مدیر سامانه را از وضعیت آگاه ساخت.



تصویر ۵۴ دریافت موفقیت‌آمیز ایمیل با عنوان "Thermal Alert" در صندوق ورودی



تصویر ۵۵ محتوای فایل پیوست شده در ایمیل هشدار حرارتی که نشان‌دهنده ثبت دقیق دمای 65.2 درجه و اعلام کاهش خودکار فریم‌ریت است